Facultatea de Electronică,
Telecomunicații și
Tehnologia Informației

Horea-Răzvan MORAR

PROIECT DE DIPLOMĂ

Sentiment Analysis of Movie Reviews using Fine-tuned BERT

SPECIALIZAREA TEHNOLOGII ȘI SISTEME DE TELECOMUNICAȚII

ÎN LIMBA ENGLEZĂ

FACULTATEA DE ELECTRONICĂ, TELECOMUNICAȚII
ȘI TEHNOLOGIA INFORMAȚIEI
UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

2026

Str. George Barițiu, nr. 26-28, 400027, Cluj-Napoca, tel: 0264-401224, tel/fax: 0264-591689, etti.utcluj.ro

UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA
FACULTATEA DE ELECTRONICĂ
TELECOMUNICAȚII ȘI TEHNOLOGIA INFORMAȚIEI
Specializarea:

Proiect de diplomă

Absolvent,

Decan,
Prof.dr.ing. Ovidiu POP

Președinte comisie,

UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA
FACULTATEA DE ELECTRONICĂ
TELECOMUNICAȚII ȘI TEHNOLOGIA INFORMAȚIEI

Departamentul

Titlul proiectului de diplomă:

Descrierea temei:

Locul de realizare:

Data emiterii temei:

Data predării temei:

Absolvent,

Director departament,

Conducător,

Benea

Declaraţie pe proprie răspundere privind autenticitatea lucrării de diplomă

Subsemnata/ul
legitimat cu seria nr. (conform copiei anexate prezentei declarații, copie semnată și certificată "conform cu originalul"), autorul lucrării

elaborată în vederea susţinerii examenului de finalizare a studiilor de licență,
la Facultatea de Electronică, Telecomunicații și Tehnologia Informației
Programul de studiu
din cadrul Universității Tehnice din Cluj-Napoca
sesiunea a anului universitar 2025-2026

declar pe proprie răspundere că această lucrare este rezultatul propriei activităţi intelectuale, pe baza cercetărilor mele şi pe baza informaţiilor obţinute din surse care au fost citate în textul lucrării şi în bibliografie.

Declar că această lucrare nu conţine porţiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislaţiei române şi a convenţiilor internaţionale privind drepturile de autor. Declar, de asemenea, că această lucrare nu a mai fost prezentată în faţa unei alte comisii de examen de absolvire/licenţă/diplomă/disertaţie.

În cazul constatării ulterioare a unor declaraţii false, voi suporta sancţiunile administrative, respectiv anularea examenului de diplomă.

Sunt de acord ca, pe tot parcursul vieții, în cazul în care este necesar și se va dori verificarea autenticității lucrării mele să fiu identificat și verificat în baza datelor declarate de mine și conform copiei documentului de identitate menționat mai sus.

Data

Nume și prenume

Semnătura



Absolvent:

Conducător:

SINTEZA PROIECTULUI DE DIPLOMĂ

Avizul conducătorului

Conducător,



Absolvent,



Contents

1. Rezumat in limba română 3
1.1 Introducere 3
1.2 Setul de date 3
1.3 Fundamente teoretice 4
1.3.1 Rețele neuronale artificiale 4
1.3.2 Arhitectura Transformer 5
1.3.3 BERT 6
1.4 Implementare 6
1.4.1 Date și preprocesare 6
1.4.2 Configurarea modelului 7
1.4.3 Antrenament și selecția modelului 7
1.5 Rezultate experimentale 8
1.5.1 Rezultatele antrenamentului 8
1.5.2 Experimentul de comparație MAX_LENGTH 8
1.5.3 Evaluarea finală pe setul de test 9
1.5.4 Analiza matricei de confuzie și a erorilor 9
1.6 Concluzii 10
2. Work planning 11
3. State of the Art 12
3.1 Evolution of Sentiment Analysis: From Classical Methods to Deep Learning 12
3.2 The Transformer Architecture and the Emergence of BERT 13
3.3 Fine-Tuning BERT for Sequence Classification: Methodology and Best Practices 13
3.4 BERT-Based Approaches for Sentiment Classification on IMDB 14
3.5 Positioning and Contribution of This Work 15
4. Theoretical Fundamentals 16
4.1 Natural Language Processing 16
4.2 Sentiment Analysis 17
4.3 Neural Networks Fundamentals 18
4.3.1 The Artificial Neuron 18
4.3.2 Activation Functions 19
4.3.3 Feedforward Neural Networks and Layers 19
4.3.4 Loss Function and Optimization 19
4.3.5 Backpropagation 20
4.4 The Transformer Architecture 20
4.4.1 The Self-Attention Mechanism 21
4.4.2 Multi-Head Attention 22
4.4.3 Positional Encoding 22
4.4.4 Position-wise Feed-Forward Network 23
4.4.5 Residual Connections and Layer Normalization 23
4.5 BERT Architecture 24
4.5.1 Model Architecture 24

4.5.2 Input Representation 24
4.5.3 Pre-Training 25
4.5.4 Fine-Tuning for Sentiment Classification 25
4.6 Evaluation Metrics 26
5. Implementation 28
5.1 Technologies and Tools 28
5.2 Training and Evaluation branch 29
5.2.1 Training Branch 30
5.2.2 Inference Branch 30
5.3 Dataset and Preprocessing 30
5.3.1 Dataset 30
5.3.2 Preprocessing 31
5.4 Model Configuration 33
5.5 Training 35
5.6 MAX_LENGTH Comparison experiment 36
5.7 Evaluation 37
5.8 User Interface 39
6. Experimental Results 41
6.1 Training Results 41
6.2 MAX_LENGTH Comparison 42
6.3 Final Evaluation Results 43
6.4 Prediction Analysis 45
6.4.1 Confusion Matrix 45
6.4.2 Error Analysis 45
6.4.3 Model Confidence Analysis 48
7. Conclusion 50
8. References 52
9. Annexes 54

1. Rezumat in limba română

1.1 Introducere

Analiza sentimentelor reprezintă una dintre sarcinile fundamentale ale procesării limbajului natural, constând în identificarea și clasificarea automată a opiniilor exprimate în text ca pozitive sau negative. Odată cu proliferarea rapidă a conținutului generat de utilizatori pe internet, incluzând recenzii de produse, postări pe rețele sociale și recenzii de film, nevoia de sisteme automate capabile să proceseze și să interpreteze opinii la scară largă a crescut semnificativ. Setul de date IMDB Large Movie Review Dataset, care conține 50.000 de recenzii de film notate cu etichete binare de sentiment, a devenit unul dintre cele mai utilizate repere pentru evaluarea sistemelor de analiză a sentimentelor, oferind un cadru standardizat de evaluare.

Abordările clasice ale analizei sentimentelor, bazate pe metode lexicale și modele de tip bag-of-words combinate cu clasificatoare tradiționale precum Support Vector Machines sau Naive Bayes, au dominat domeniul în primii ani de dezvoltare. Deși eficiente din punct de vedere computațional și interpretabile, aceste metode prezintă limitări fundamentale: tratează textul ca o colecție neordonată de tokeni, ignorând ordinea cuvintelor, structura sintactică și dependențele semantice dintre termeni. Ca urmare, performanța lor se degradează semnificativ pe texte lungi, cum ar fi recenziile de film, unde sentimentul este frecvent exprimat prin structuri argumentative complexe, ironie și vocabular specific domeniului.

Apariția modelelor de reprezentare distribuită a cuvintelor, precum Word2Vec și GloVe, urmată de arhitecturile de rețele neuronale recurente, în special Long Short-Term Memory (LSTM), a adus îmbunătățiri substanțiale față de metodele clasice, atingând acurateță de 85–89% pe setul de date IMDB [7]. Cu toate acestea, natura secvențială a procesării LSTM limitează paralelizarea în timpul antrenamentului și scalabilitatea la secvențe foarte lungi, reprezentând o limitare practica pentru texte de dimensiuni mari.

Introducerea arhitecturii Transformer în 2017 de către Vaswani et al. și a modelului BERT în 2019 de către Devlin et al. a reprezentat un punct de inflexiune în domeniu. Prin mecanismul de self-attention bidirecțional, care permite fiecărui token să acceseze direct contextul tuturor celorlalți tokeni din secvență indiferent de distanța poziționlă, și prin strategia de pre-antrenare pe corpusuri masive de text prin obiectivele Masked Language Modeling și Next Sentence Prediction, BERT a stabilit noi standarde de performanță pe numeroase repere de procesare a limbajului natural. Prin fine-tuning pe date specifice sarcinii, modelul pre-antrenat poate fi adaptat eficient la clasificarea sentimentelor cu un număr relativ mic de epoci de antrenament.

Lucrarea de față prezintă dezvoltarea și evaluarea unui sistem de analiză a sentimentelor bazat pe fine-tuning-ul modelului bert-base-uncased pe setul de date IMDB, în cadrul unui pipeline complet documentat și reproductibil. Configurația de fine-tuning include optimizatorul AdamW cu rată de învățare de 2×10^{-5}, program liniar de încălzire, gradient clipping și lungimea maximă a secvenței de 512 tokeni,. Modelul final a obținut o acurateță de 94.50% și un scor F1 de 94.56% pe setul de test, rezultate superioare celor raportate în studiile recente selectate pentru comparație în această lucrare.

1.2 Setul de date

Setul de date utilizat în această lucrare este IMDB Large Movie Review Dataset, introdus de Maas et al. (2011), care conține 50.000 de recenzii de film în limba engleză, colectate de pe Internet Movie Database și notate cu etichete binare de sentiment: pozitiv sau negativ. Setul de date este echilibrat prin construcție, conținând exact 25.000 de recenzii pozitive și 25.000 de recenzii negative, ceea ce elimină necesitatea aplicării unor tehnici de re-eșantionare în timpul

antrenamentului. Datorită dimensiunii sale și caracterului standardizat, IMDB a devenit unul dintre cele mai utilizate reperuri pentru clasificarea binară a sentimentelor.

O analiză exploratorie a setului de date a relevat absența valorilor lipsă și a câmpurilor de text goale. Analiza lungimii recenziilor la nivel de caractere a evidențiat o medie de 1.289 de caractere, o mediană de 957 și un maxim de 13.593 de caractere. Această distribuție indică faptul că o proporție semnificativă a recenziilor depășește fereastra de context de 512 tokeni a modelului BERT, făcând din alegerea strategiei de trunchiere o decizie relevantă pentru etapa de preprocesare. Analiza a identificat, de asemenea, prezența tag-urilor HTML, a artefactelor de spațiu alb și a unui număr de înregistrări duplicate, toate rezolvate în etapa ulterioară de curățare.

1.3 Fundamente teoretice

1.3.1 Rețele neuronale artificiale

Baza teoretică a modelelor moderne de procesare a limbajului natural, inclusiv BERT, o constituie rețelele neuronale artificiale, inspirate din structura și funcționarea neuronilor biologici din creierul uman. Unitatea computațională fundamentală este neuronul artificial, care calculează o sumă ponderată a intrărilor sale și aplică o funcție de activare neliniară rezultatului. Dat un vector de intrare x, un vector de ponderi W și un termen de bias b, pre-activarea z este calculată conform formulei (1):

$$z = W \cdot x + b \tag{1}$$

Rezultatul z este apoi trecut printr-o funcție de activare f(·), producând ieșirea neuronului a = f(z).

Funcțiile de activare joacă un rol esențial în arhitecturile neuronale, introducând neliniaritatatea necesară pentru ca rețelele cu mai multe straturi să poată aproxima funcții arbitrar de complexe. Funcția Sigmoid mapează orice valoare reală în intervalul (0,1), conform formulei (2):

$$\sigma(z) = \frac{1}{(1+e^{-z})} \tag{2}$$

Unitatea Liniară Rectificată (ReLU) returnează direct intrarea pentru valori pozitive și zero altfel, conform formulei (3):

$$f(z) = \max(0, z) \tag{3}$$

Funcția Softmax este utilizată la nivelul stratului de ieșire pentru a converti un vector de scoruri brute în o distribuție de probabilitate validă peste K clase, conform formulei (4):

$$softmax(z)_i = \frac{e^{zi}}{\sum_j e^{zj}}, i = 1, \ldots., K \tag{4}$$

Neuronii individuali sunt organizați în straturi pentru a forma o rețea neuronală feedforward, numită și Perceptron Multi-Strat (MLP). Transformarea aplicată de un singur strat este exprimată prin formula (5):

$$a^{(l)} = f\left(W^{(l)}a^{(l-1)} + b^{(l)}\right) \quad (5)$$

Antrenamentul rețelei se realizează prin algoritmul de backpropagation, care aplică regula lanțului din calcul diferențial pentru a calcula gradientul funcției de pierdere față de fiecare parametru al rețelei, utilizând ulterior un algoritm de optimizare de tip gradient descent pentru actualizarea parametrilor.

1.3.2 Arhitectura Transformer

Arhitectura Transformer, introdusă de Vaswani et al. în 2017 în lucrarea "Attention is All You Need", a înlocuit recurența cu un mecanism de self-attention capabil să modeleze dependențele globale dintre tokeni în paralel. Spre deosebire de arhitecturile LSTM, care procesează tokenii secvențial, Transformer-ul procesează întreaga secvență simultan, eliminând blocajul de procesare secvențială și permițând paralelizarea completă în timpul antrenamentului.

Mecanismul central al arhitecturii este scaled dot-product attention, care transformă intrarea în trei proiecții liniare: matricea Query (Q), matricea Key (K) și matricea Value (V). Atenția ponderată este calculată conform formulei (6):

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (6)$$

unde factorul de scalare $\frac{1}{\sqrt{\mathrm{d_k}}}$ previne saturarea funcției softmax pentru produse scalare de magnitudine mare. Pentru a captura simultan informații din multiple subspații de reprezentare, arhitectura utilizează Multi-Head Attention, calculând h funcții de atenție în paralel și concatenând rezultatele, conform formulei (7):

$$MultiHead(Q, K, V) = Concat(head_1, head_2, \ldots., head_h)W^O \quad (7)$$

Deoarece mecanismul de self-attention este invariant la permutări, informația pozițională este injectată explicit prin codificare pozițională sinusoidală, adăugată la embedding-urile de token la intrarea în encoder, conform formulelor (8) si (9):

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right) \quad (8)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right) \quad (9)$$

Fiecare strat encoder conține, de asemenea, o rețea feed-forward poziție cu poziție, conexiuni reziduale și normalizare de strat, toate contribuind la stabilitatea antrenamentului și la capacitatea modelului de a construi reprezentări abstracte și ierarhice ale textului.

1.3.3 BERT

BERT (Bidirectional Encoder Representations from Transformers), introdus de Devlin et al. în 2019, reține exclusiv componenta encoder a arhitecturii Transformer și îl pre-antrenează pe corpusuri masive de text printr-o strategie bidirecțională completă. Reprezentarea de intrare a BERT este construită prin suma a trei tipuri de embedding-uri pentru fiecare token, conform formulei (10):

$$E_i = TokenEmbedding(t_i) + SegmentEmbedding(s_i) + PositionEmbedding(i) \quad (10)$$

Pre-antrenarea BERT se realizează prin două obiective complementare. Primul, Masked Language Modeling (MLM), maschează aleatoriu 15% din tokenii de intrare și antrenează modelul să le reconstruiască pe baza contextului bilateral, conform formulei (11):

$$L_{MLM} = -logP(x_i|\tilde{x}) \quad (11)$$

Al doilea obiectiv, Next Sentence Prediction (NSP), antrenează modelul să determine dacă două propoziții apar consecutiv în corpusul original. Pierderea combinată de pre-antrenare este suma celor două componente, conform formulei (12):

$$L_{pre-train} = L_{MLM} + L_{NSP} \quad (12)$$

Pentru clasificarea sentimentelor, BERT este adaptat prin fine-tuning: un cap de clasificare liniar este adăugat pe reprezentarea tokenului [CLS] produs de ultimul strat encoder, iar întregul model este optimizat end-to-end pe date etichetate, minimizând pierderea de entropie încrucișată, conform formulei (13):

$$L_{fine-tune} = -\sum_{c=0}^{1} y_c \log(\hat{y}_c) \quad (13)$$

1.4 Implementare

1.4.1 Date și preprocesare

Implementarea a utilizat Python 3.10.20 într-un mediu virtual Anaconda, cu antrenamentul executat pe Google Colab folosind un GPU NVIDIA T4. Librăriile principale utilizate sunt PyTorch pentru infrastructura de deep learning, Transformers și Datasets pentru încărcarea și gestionarea modelului și datelor, scikit-learn pentru împărțirea datelor și calculul metricilor, matplotlib pentru vizualizări și Gradio pentru interfața web.

Setul de date a fost supus unui pipeline de preprocesare în cinci etape. În prima etapă, datele brute au fost încărcate și cele două split-uri implicite au fost combinate într-un singur DataFrame pentru a permite o partiționare complet personalizată. Această concatenare a fost necesară pentru a aplica etapele de curățare și deduplicare uniform pe întregul set de date, înainte de împărțirea în subseturi.

În a doua etapă, textul brut a fost curățat prin eliminarea tag-urilor HTML folosind expresii regulate și normalizarea spațiilor albe. Această etapă a fost urmată de o deduplicare care a redus numărul de recenzii de la 50.000 la 49.578, eliminând o potentiala sursa de scurgere a informatiei intre subseturi.

În a treia etapă, setul de date curățat a fost împărțit în trei subseturi disjuncte folosind o strategie de împărțire stratificată în două etape, cu proporțiile 70% antrenament, 15% validare și

15% test, obținând 34.704 exemple de antrenament, 7.437 de validare și 7.437 de test. O sămânță aleatoare fixă de 42 a fost aplicată pe parcursul întregului pipeline pentru a asigura reproducibilitatea completă. Setul de test a fost rezervat exclusiv pentru evaluarea finală și nu a fost consultat în niciun moment al procesului de dezvoltare a modelului.

În a patra etapă, tokenizarea textului a fost realizată cu BertTokenizer asociat checkpoint-ului bert-base-uncased. Fiecare recenzie a fost tokenizată cu adăugarea tokenilor speciali [CLS] și [SEP] impuși de BERT. Secvențele mai lungi decât lungimea maximă au fost trunchiate, iar cele mai scurte au fost completate cu padding la o lungime uniformă pentru a permite procesarea în batch-uri.

În a cincea și ultima etapă, dataset-ul tokenizat a fost salvat pe disc în formatul DatasetDict, evitând recomputarea redundantă a pipeline-ului de tokenizare în rulările ulterioare.

1.4.2 Configurarea modelului

Modelul selectat este bert-base-uncased, un encoder Transformer pre-antrenat cu 12 straturi de atenție, 12 capete de atenție, o dimensiune ascunsă de 768 și aproximativ 110 milioane de parametri. Varianta base a fost preferată față de bert-large deoarece câștigurile de performanță oferite de varianta mai mare nu justifică cerințele substanțial mai mari de memorie și timp de antrenament pentru o sarcină binară bine definită. Varianta uncased a fost aleasă deoarece polaritatea sentimentului în recenziile de film nu este codificată sistematic prin majuscule.

Optimizatorul utilizat este AdamW, cu o rată de învățare de 2×10^{-5} și un coeficient de weight decay de 0,01. Un program liniar de încălzire pe primii 10% din pașii de antrenament, urmat de o scădere liniară la zero, a fost aplicat pentru a stabiliza fazele inițiale ale fine-tuning-ului. Gradient clipping cu norma maximă de 1,0 a prevenit actualizările de gradient excesiv de mari. Antrenamentul a fost condus pentru 3 epoci cu o dimensiune de batch de 16. Configurația completă a hiperparametrilor este prezentată în Tabelul 1.

Tabelul 1. Configurația hiperparametrilor utilizați pentru fine-tuning BERT

Hiperparametru	Valoare
Model	bert-base-uncased
MAX_LENGTH	512
Dimensiune batch	16
Număr epoci	3
Optimizator	AdamW
Rată de învățare	2×10^{-5}
Weight decay	0.01
Pași de încălzire	10% of total steps
Gradient clipping	1.0
Sămânță aleatoare	42

1.4.3 Antrenament și selecția modelului

Pipeline-ul de antrenament a fost executat pe Google Colab cu GPU NVIDIA T4. Înainte de bucla de antrenament, dataset-ul tokenizat a fost încărcat de pe disc și împachetat în obiecte DataLoader pentru fiecare subset. DataLoader-ul de antrenament a utilizat amestecare pentru a asigura o ordine diferită a batch-urilor la fiecare epocă, în timp ce DataLoader-urile de validare și test au păstrat o ordine fixă.

La fiecare epocă, modelul a fost setat în modul de antrenament și toate batch-urile din DataLoader-ul de antrenament au fost procesate secvențial. Pentru fiecare batch, tensorii de intrare

au fost mutați pe GPU, a fost realizată o trecere înainte pentru a calcula pierderea, iar backpropagation a fost aplicat pentru a calcula gradienții. Gradient clipping a fost aplicat înainte de pasul optimizatorului, iar scheduler-ul ratei de învățare a fost actualizat după fiecare batch.

La sfârșitul fiecărei epoci, modelul a fost evaluat pe setul de validare. Dacă epoca curentă a produs o acuratețe de validare mai mare decât cel mai bun rezultat anterior, ponderile modelului au fost salvate pe disc ca cel mai bun checkpoint. Selecția modelului a fost realizată exclusiv pe baza performanței pe setul de validare, testul final nefiind consultat în niciun moment al acestui proces.

1.5 Rezultate experimentale

1.5.1 Rezultatele antrenamentului

Modelul BERT fine-tuned a fost antrenat pentru 3 epoci pe setul de antrenament de 34.704 exemple, cu validare la sfârșitul fiecărei epoci. Rezultatele sunt prezentate în Tabelul 2.

Tabelul 2. Rezultatele antrenamentului pe epoci

Epocă	Train Loss	Train Accuracy	Val Loss	Val Accuracy
1	0.2771	88.35%	0.1652	93.79%
2	0.1425	95.84%	0.2563	93.69%
3	0.0746	98.17%	0.2540	94.62%

Pierderea de antrenament a scăzut consistent pe parcursul celor trei epoci, de la 0,2771 la epoca 1 la 0,0746 la epoca 3. Acuratețea de validare a început la 93,79%, a scăzut ușor la 93,69% la epoca 2, iar apoi a crescut la 94,62% la epoca 3. Cel mai bun checkpoint al modelului a fost salvat la epoca 3. Diferența dintre acuratețea de antrenament (98,17%) și cea de validare (94,62%) este așteptată și nu indică un overfitting sever, dat fiind că performanța de validare a rămas ridicată pe parcursul întregului antrenament.

1.5.2 Experimentul de comparație MAX_LENGTH

Pentru a determina lungimea optimă a secvenței de tokenizare, trei configurații au fost evaluate pe setul de validare: 128, 256 și 512 tokeni. Rezultatele sunt prezentate în Tabelul 3.

Tabelul 3. Rezultatele comparației MAX_LENGTH pe setul de validare

MAX_LENGTH	Val Accuracy	Val F1 Score	Epoch 1 time	Epoch 2 time	Epoch 3 time	Timp total antrenament
128	89.90%	90.06%	15.2 min	15.4 min	15.4 min	50.9 min
256	92.77%	92.80%	30 min	29.9 min	30 min	97.4 min
512	94.26%	94.33%	53.8 min	53.8 min	53.8 min	174.1 min

Atât acuratețea de validare cât și scorul F1 cresc consistent odată cu creșterea lungimii maxime a secvenței. Îmbunătățirea de la 128 la 256 tokeni este mai mare decât cea de la 256 la 512 tokeni, sugerând randamente descrescătoare pe măsură ce lungimea secvenței se apropie de maximul suportat de BERT. Motivul principal pentru performanța superioară a secvențelor mai lungi este natura setului de date IMDB: recenziile sunt adesea lungi și detaliate, cu sentimentul exprimat pe parcursul mai multor paragrafe. Creșterea timpului de antrenament de la 50,9 minute pentru MAX_LENGTH=128 la 174,1 minute pentru MAX_LENGTH=512 a fost considerată

acceptabilă în raport cu câștigul de performanță. Pe baza acestor rezultate, MAX_LENGTH=512 a fost selectat ca configurație finală.

1.5.3 Evaluarea finală pe setul de test

Modelul final, antrenat cu MAX_LENGTH=512, a fost evaluat o singură dată pe setul de test de 7.437 exemple. Rezultatele sunt prezentate în Tabelul 4

Tabelul 4. Rezultatele evaluării finale pe setul de test

Metrică	Valoare
Acuratețe	94.50%
Precizie	93.90%
Recall	95.23%
Scor F1	94.56%

Modelul a obținut o acuratețe de 94.50% și un scor F1 de 94.56% pe setul de test. Acordul strâns dintre acuratețe și scorul F1 confirmă că modelul performează consistent pe ambele clase, fără un bias semnificativ față de predicțiile pozitive sau negative. Diferența mică dintre precizie (93,90%) și recall (95,23%) indică faptul că modelul este ușor mai eficient în detectarea exemplelor pozitive decât în evitarea fals pozitivelor.

Aceste rezultate sunt superioare celor raportate în studiile recente selectate pentru comparație în această lucrare. Puspita și Rahayu au raportat o acuratețe de 91.78% cu un model BERT-base fine-tuned, Islam et al. au raportat BERT ca cea mai performantă arhitectură în comparația lor cu o acuratețe sub 92%, iar Popescu et al. au raportat o acuratețe BERT de 92% și un scor F1 de 92%. Modelul dezvoltat în această lucrare obține 94.50%, obtinand o performanta superioara lucrarilor comparate.

1.5.4 Analiza matricei de confuzie și a erorilor

Matricea de confuzie obținută pe setul de test este prezentată în Figura 1. Din cele 7.437 de exemple de test, modelul a clasificat corect 3.474 de recenzii negative (True Negatives) și 3.554 de recenzii pozitive (True Positives). A clasificat incorect 231 de recenzii negative ca pozitive (False Positives) și 178 de recenzii pozitive ca negative (False Negatives).

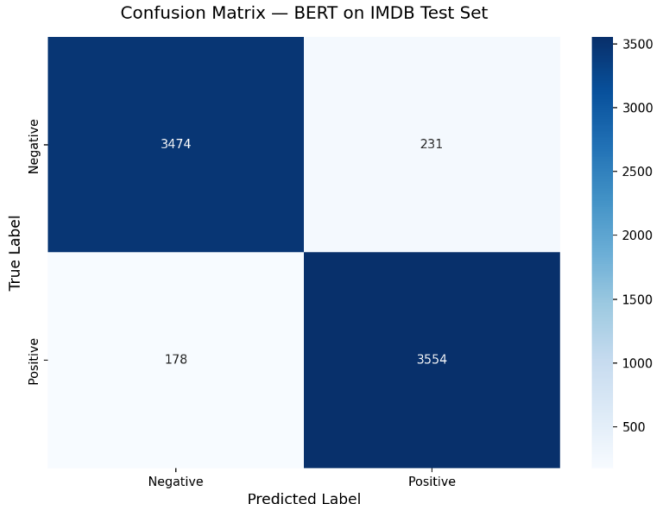


Figura 1. Matricea de confuzie BERT pe setul de test IMDB

Distribuția erorilor este relativ echilibrată între cele două clase. Numărul ușor mai mare de fals pozitive (231) față de fals negative (178) sugerează că modelul este marginal mai predispus să prezică sentimentul pozitiv, dar diferența este mică și nu afectează fiabilitatea generală a rezultatelor.

Pentru a înțelege mai bine unde face modelul greșeli, a fost realizată o analiză a erorilor pe cele 409 recenzii clasificate incorect, reprezentând o rată de eroare de 5,50%. Prima observație privește lungimea recenziilor: recenziile clasificate greșit sunt în medie mai lungi decât cele clasificate corect, cu o lungime medie de 243 de cuvinte față de 228 de cuvinte. Acest lucru sugerează că recenziile mai lungi, mai susceptibile să fie trunchiate la 512 tokeni, sunt mai dificil de clasificat corect.

A doua observație privește ironia și sarcasmul: recenziile care conțin cuvinte cheie asociate ironiei au prezentat o rată de clasificare incorectă de 5,8%, ușor mai mare decât rata generală de 5,50%. Când o recenzie folosește limbaj indirect, ironie sau sarcasm, cuvintele individuale pot purta un sentiment diferit față de semnificația globală intenționată de autor. Modelul poate întâmpina dificultăți în aceste cazuri deoarece nu a văzut suficiente exemple de acest tip de limbaj în timpul antrenamentului.

1.6 Concluzii

Această lucrare a prezentat dezvoltarea și evaluarea unui sistem de analiză a sentimentelor bazat pe fine-tuning-ul modelului BERT pe setul de date IMDB Large Movie Review Dataset. Obiectivul principal a fost construirea unui pipeline bine documentat și reproductibil care să atingă performanțe superioare celor raportate în literatura recentă, respectând cele mai bune practici în ceea ce privește împărțirea datelor, selecția modelului și evaluarea rezultatelor.

Modelul a obținut o acuratețe de 94,50% și un scor F1 de 94,56% pe setul de test, rezultate superioare celor raportate în studiile recente selectate pentru comparație. Analiza erorilor a evidențiat mai multe tipare în exemplele clasificate incorect. Recenziile clasificate greșit au fost, în medie, mai lungi decât cele clasificate corect, iar analiza calitativă a unor exemple individuale a evidențiat dificultăți în cazul recenziilor cu sentiment mixt, formulări indirecte sau indicia contradictorii. Aceste observații indică posibile direcții pentru investigarea limitărilor modelului.

Lucrările viitoare ar putea investiga mai riguros influența lungimii și a trunchierii prin compararea performanței pe recenzii trunchiate și netrunchiate sau prin utilizarea unor modele cu o fereastră de context mai lungă, precum Longformer sau BigBird. De asemenea, ar putea fi extinsă analiza pe categorii de dificultate definite sistematic, precum sentimental mixt, formulările indirecte și prezența indiciilor contradictorii. Metodele de tip ansamblu, care combină mai multe modele transformer, ar putea reprezenta o direcție suplimentară pentru îmbunătățirea performanței.

2. Work planning

	Task	Start of date	No. of days	End date
1	Topic selection and documentation on the sentiment analysis domain	24.10.2025	2	26.10.2025
2	Survey of the state-of-the-art approaches for sentiment classification on the IMDB dataset	27.10.2025	32	28.11.2025
3	Study of the Transformer architecture and the BERT model. Understanding the fine-tuning mechanism for classification tasks	29.11.2025	13	12.12.2025
4	Collection and exploratory data analysis of the IMDB dataset. Identification of data quality issues	13.12.2025	60	11.02.2026
5	Implementation of the preprocessing pipeline: text cleaning, deduplication, stratified splitting and tokenization	12.02.2026	20	04.03.2026
6	Model configuration: instantiation of bert-base-uncased with a classification head and implementation of the training loop	05.03.2026	25	30.03.2026
7	Model training on Google Colab with NVIDIA T4 GPU. Best checkpoint selection based on validation set performance	31.03.2026	20	20.04.2026
8	MAX_LENGTH comparison experiment. Evaluation of configurations 128, 256 and 512 tokens on the validation set	21.04.2026	10	01.05.2026
9	Final model evaluation on the test set. Computation of metrics and generation of the confusion matrix	02.05.2026	12	14.05.2026
10	Error analysis. Identification of misclassification patterns: review length and irony/sarcasm	15.05.2026	8	23.05.2026
11	Development of the web interface with Gradio and deployment on Hugging Face Spaces	24.05.2026	13	07.06.2026
12	Writing of the diploma thesis	08.06.2026	28	6.07.2026

3. State of the Art

3.1 Evolution of Sentiment Analysis: From Classical Methods to Deep Learning

Sentiment analysis is the computational task of identifying, extracting, and categorizing subjective information from text, with the primary objective of determining the polarity of expressed opinions: whether positive, negative, or neutral [1]. The field emerged in the early 2000s alongside the rapid proliferation of user-generated content on the internet, including product reviews, social media posts, forum discussions, and movie ratings, which created both the need and the opportunity for automated opinion processing at scale [1].

The earliest approaches to sentiment analysis were grounded in lexicon-based methods, which relied on predefined dictionaries of sentiment-bearing words and handcrafted linguistic rules to assign polarity scores to text. While computationally inexpensive and interpretable, these methods were inherently brittle: they could not adapt to new domains, failed to handle negation and context-dependent expressions, and depended heavily on the coverage and quality of the underlying lexicons [2]. Their performance degraded significantly on datasets such as movie reviews, where sentiment is frequently expressed through complex rhetorical devices, irony, and domain-specific vocabulary [1].

The transition to statistical machine learning brought more flexible and data-driven solutions. Classifiers such as Naive Bayes, Logistic Regression, and Support Vector Machines (SVM) were applied to sentiment classification tasks using manually engineered feature representations, most notably bag-of-words (BoW) models and term frequency-inverse document frequency (TF-IDF) vectors [2]. SVM in particular demonstrated strong performance on binary polarity classification, and for many years remained the baseline against which more complex systems were measured. However, these methods shared a fundamental limitation: they treated text as unordered collections of tokens, discarding all information about word order, syntactic structure, and semantic relationships between terms [2]. As a consequence, they struggled with long-form documents such as movie reviews, where sentiment may shift across paragraphs and where the overall polarity cannot be reliably inferred from local word statistics alone.

The advent of deep learning in the early 2010s introduced representation learning capabilities that addressed many of these shortcomings. Word embedding models such as Word2Vec [10] and GloVe [11] enabled words to be mapped to dense continuous vector spaces in which semantic similarity is captured through geometric proximity. These distributed representations were subsequently used as inputs to neural sequence models. Recurrent Neural Networks (RNN) [3], and in particular Long Short-Term Memory (LSTM) [3] networks were designed to process sequential inputs while maintaining a hidden state that accumulates contextual information over time, making them well-suited for capturing dependencies across long text sequences [3]. Convolutional Neural Networks (CNN), while originally developed for image recognition, were adapted for text classification by treating feature maps over local n-gram windows, enabling efficient extraction of local patterns [3]. Studies conducted on the IMDB movie reviews dataset confirmed that LSTM and CNN architectures consistently outperformed classical machine learning classifiers, achieving accuracy values in the range of 88–91% [3]. Nevertheless, both architectures exhibited practical limitations: LSTM training was computationally expensive and prone to vanishing gradients over very long sequences, while CNNs, despite their efficiency, lacked the capacity to model long-range dependencies within documents [4].

3.2 The Transformer Architecture and the Emergence of BERT

A fundamental breakthrough in natural language processing was achieved in 2017 with the introduction of the Transformer architecture by Vaswani et al.[12], which replaced recurrence with a multi-head self-attention mechanism capable of modelling global dependencies within sequences in parallel, enabling full parallelization during training and eliminating the gradient propagation problems associated with recurrent architectures [12].

Building directly on the Transformer encoder, Devlin et al. (2019) at Google AI introduced BERT (Bidirectional Encoder Representations from Transformers), which constituted a landmark advance in the field [5]. Unlike prior language models that processed text unidirectionally, BERT introduced a fully bidirectional pre-training strategy through two novel objectives: Masked Language Modeling (MLM), which randomly masks 15% of input tokens and trains the model to predict them from surrounding context, and Next Sentence Prediction (NSP), which trains the model to determine whether two sentences appear consecutively in the original corpus [5]. BERT was released in two configurations: BERT-Base, with 12 Transformer encoder layers, 12 self-attention heads, a hidden dimension of 768, and approximately 110 million parameters; and BERT-Large, with 24 layers, 16 attention heads, a hidden dimension of 1024, and approximately 340 million parameters [5]. Both variants were pre-trained on approximately 3.3 billion words drawn from English Wikipedia and the BooksCorpus, and at publication achieved new state-of-the-art results across eleven NLP benchmarks, including a GLUE score of 80.5% [5].

Following BERT, derivative models such as RoBERTa [13], DistilBERT [14], and XLNet [15] were proposed to address specific limitations regarding training efficiency, model size, and pre-training objectives respectively. RoBERTa improved upon BERT by removing the NSP objective and training on significantly larger corpora, while DistilBERT reduced model size by 40% and improved inference speed by 60% while retaining approximately 97% of BERT's performance, and XLNet introduced a permutation-based language modelling objective that avoids the mismatch between pre-training and fine-tuning introduced by the [MASK] token [6].

3.3 Fine-Tuning BERT for Sequence Classification: Methodology and Best Practices

The fine-tuning of BERT for downstream classification tasks follows a well-established procedure that has been refined through extensive empirical investigation in the literature [5][9]. For binary sentiment classification, a linear layer is appended to the [CLS] token representation produced by the final encoder layer, and the entire model, including all 110 million parameters of the pre-trained encoder is updated jointly on the labelled training set. This end-to-end fine-tuning strategy, rather than feature extraction, has been consistently shown to yield superior results [5].

A critical aspect of successful BERT fine-tuning is the choice of optimization algorithm and learning rate schedule. The AdamW optimizer, which extends the Adam optimizer with decoupled weight decay regularization, is the standard choice for transformer fine-tuning and is recommended in the original BERT paper [5]. Empirical studies have established that learning rates in the range of 2×10^{-5} to 5×10^{-5} are appropriate for most classification tasks, with values significantly higher than this range leading to catastrophic forgetting of the pre-trained representations [9]. A linear warmup schedule over the initial 10% of training steps, followed by linear decay to zero, has been shown to stabilize the early phase of training by gradually increasing the learning rate before the model's parameters have adapted to the new task [9]. Gradient clipping with a maximum norm of 1.0 is additionally recommended to prevent gradient explosions that can occur when the loss landscape exhibits sharp curvatures during fine-tuning [9]. Weight decay of 0.01 is typically applied to all parameters except bias terms and layer normalization parameters, consistent with the original BERT fine-tuning recommendations [5].

The choice of maximum sequence length is another important consideration. BERT-Base supports sequences of up to 512 tokens, but many fine-tuning studies adopt shorter lengths (e.g., 128 or 256 tokens) to reduce memory requirements and training time. In the context of movie review classification, where reviews frequently span several paragraphs and sentiment may be expressed through complex argumentative structures, the use of the full 512-token context has been empirically shown to provide meaningful accuracy gains over shorter truncated representations [3]. The batch size and number of training epochs also influence final performance: Devlin et al. recommend batch sizes of 16 or 32 and training for 2–4 epochs, as longer training schedules tend to result in overfitting on tasks with large labelled datasets [5].

3.4 BERT-Based Approaches for Sentiment Classification on IMDB

Since its introduction, BERT has been extensively applied to sentiment analysis tasks, with the IMDB Large Movie Review Dataset serving as one of the most widely adopted benchmarks for binary sentiment classification. The dataset, introduced by Maas et al. (2011) [16], consists of 50,000 movie reviews equally divided between positive and negative sentiment labels, and has become a standard reference point for comparing the performance of sentiment classification systems across methodological generations.

Puspita and Rahayu (2023) applied fine-tuned BERT-base directly to the IMDB dataset and reported a test accuracy of 91.78%, with a training accuracy of 91.69% and a test loss of 0.207 [7]. Their work provided an explicit comparison against traditional classifiers: including Logistic Regression, Naive Bayes, Decision Trees, and BiLSTM, on the same dataset, confirming that BERT substantially and consistently outperforms all classical and shallow deep learning baselines [7]. The margin of improvement over BiLSTM, the strongest non-transformer baseline in their evaluation, exceeded 3 percentage points in accuracy, reflecting the qualitative advance that bidirectional pre-training brings to this task.

Islam et al. (2024) conducted a systematic comparative evaluation of CNN, LSTM, CNN-LSTM hybrid, and BERT architectures for binary sentiment classification on the IMDB 50K dataset [3]. Their experimental protocol evaluated all models under identical training and evaluation conditions, enabling a direct comparison of architectural contributions. BERT achieved the highest classification accuracy and F1-score among all evaluated architectures, with the gap between BERT and the best CNN-LSTM baseline exceeding 2 percentage points, reinforcing its suitability for large-scale sentiment analysis tasks [3].

A comprehensive study by Popescu et al. (2025) evaluated five transformer architectures: BERT, RoBERTa, DistilBERT, GPT-2, and XLNet, under identical experimental conditions on the IMDB dataset as part of the TWSSenti hybrid sentiment analysis framework [8]. In their evaluation, BERT achieved an accuracy of 92% and an F1-score of 92%, while RoBERTa reached 91% and DistilBERT 89% [8]. The study further noted that DistilBERT, despite its lower accuracy, offers a compelling efficiency trade-off for deployment in resource-constrained environments, and that GPT-2 and XLNet performed below BERT on this specific binary classification task [8].

Areshey and Mathkour (2024) performed a broader cross-dataset comparison of five transformer variants: BERT, RoBERTa, ALBERT, DistilBERT, and XLNet [6]. Their findings confirmed that transfer learning models universally and substantially outperform classical machine learning approaches across a diverse set of text classification tasks. The study additionally highlighted that the relative performance ordering among transformer variants is not consistent across datasets and tasks, and that the optimal model choice is influenced by factors including dataset size, domain specificity, and the distribution of sequence lengths in the corpus [6].

Danyal et al. (2024) proposed a hybrid sentiment analysis model combining BERT and XLNet representations for movie review classification [4]. Their work demonstrated that ensemble strategies combining complementary transformer architectures can yield accuracy improvements over individual models, achieving approximately 90% accuracy and F1-score on movie review datasets [4]. However, the increased architectural complexity introduces additional computational

cost at both training and inference time, which may limit practical applicability in latency-sensitive deployment scenarios.

3.5 Positioning and Contribution of This Work

The body of literature reviewed in the preceding sections establishes that transformer-based models, and BERT in particular, represent the current state of the art for binary sentiment classification on the IMDB dataset, with reported accuracies ranging from 89% to 92% across recent studies [3][6][7][8].

The present work addresses these limitations through a methodologically rigorous experimental design applied to the full IMDB dataset of 49,578 deduplicated reviews. A stratified three-way split of 70% training, 15% validation, and 15% test ensures that all model selection decisions, including the choice of maximum sequence length and the number of training epochs, are made exclusively on the basis of validation set performance, and that the test set is used exactly once for final evaluation. This approach eliminates the risk of implicit data leakage that can inflate reported accuracy figures in studies that select configurations based on test set feedback.

The primary objective of this work is to investigate whether a carefully fine-tuned BERT-base-uncased model, trained under well-documented and reproducible conditions, can achieve performance competitive with or superior to the results reported in the recent studies selected for comparison. The fine-tuning configuration: AdamW optimizer with learning rate 2×10^{-5}, weight decay 0.01, linear warmup over 10% of training steps, linear decay to zero, gradient clipping with maximum norm 1.0, and full 512-token sequence length, is selected on the basis of established best practices in the literature [5][9] and validated empirically through the MAX_LENGTH comparison experiment conducted on the validation set. The resulting model achieves a test accuracy score of 94.50% and F1-score of 94.56%, exceeding the results reported in the recent studies selected for comparison and providing a well-documented experimental result for this Bert-base configuration on the IMBD sentiment classification benchmark.

4. Theoretical Fundamentals

4.1 Natural Language Processing

Natural Language Processing (NLP) is a subfield of artificial intelligence which enables computing systems to effectively process, interpret, and produce human language. As illustrated in Figure 2, we can observe the five levels of linguistic processing involved in NLP, each building upon the previous one to transform raw text into structured, machine-interpretable information.

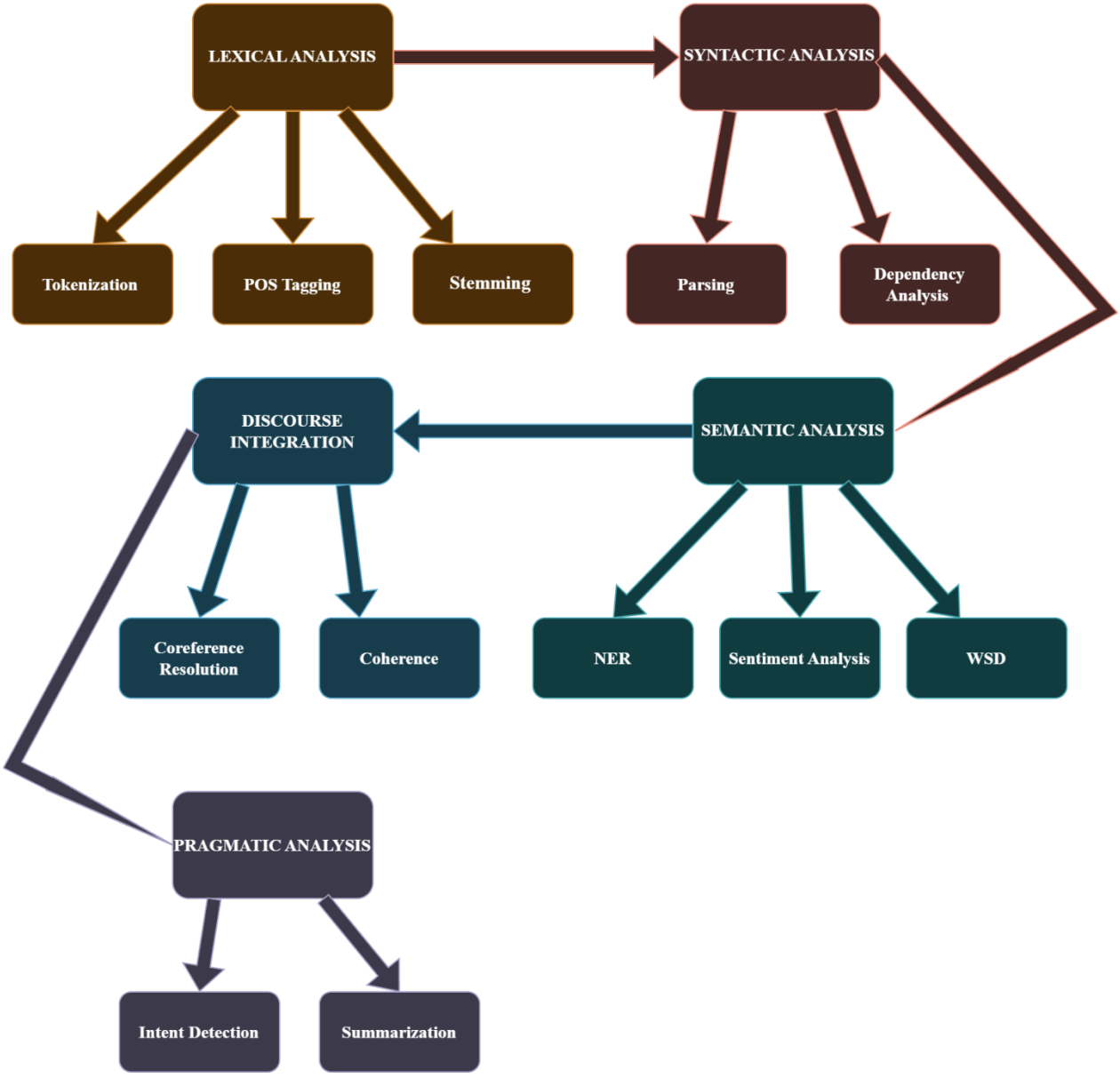


Figure 2. Levels of linguistic analysis in Natural Language Processing.

Lexical Analysis begins by breaking raw text into its basic units through processes such as tokenization, part-of-speech tagging, and stemming. It identifies individual words and their grammatical roles, supplying the structural input for all subsequent processing stages. Syntactic Analysis examines the grammatical relationships within a sentence through parsing and dependency analysis in order to construct a structural representation that reflects how words relate to one another.

Semantic Analysis attempts to interpret the significance of individual words and their interactions within text. A key component of this level is the recognition of named entities, word sense disambiguation, and sentiment analysis, which analyses a text's emotional polarity. Discourse Integration extends this understanding beyond individual sentences, resolving coreferences and analysing coherence across longer text spans such as paragraphs and documents. Lastly, Pragmatic Analysis interprets language's intent within a broader communicative context, assisting in tasks like intent detection and automatic summarization.

The present work employs a fine-tuned transformer model to analyze movie reviews at the document level, operating specifically within the semantic layer of the hierarchy to conduct binary sentiment classification. Unlike rule-based or lexicon-driven approaches that address only the lower levels of this pipeline, the BERT-based architecture employed in this work integrates information across all linguistic levels through its deep bidirectional attention mechanisms, enabling a richer and more contextually grounded understanding of sentiment in long-form text.

4.2 Sentiment Analysis

Sentiment analysis is a subfield of Natural Language Processing concerned with the computational identification and categorization of subjective information expressed in text, with the primary objective of determining the attitude, opinion, or emotional polarity of the author toward a given subject [1]. As a highly active research area within NLP, sentiment analysis has proven to be a highly versatile tool across domains including product review analysis, social media and political opinion tracking, financial market prediction, and customer feedback processing.

Fundamentally, sentiment analysis operates as a supervised classification task where an algorithm approximates a function $f: X \rightarrow Y$, where X represents the input text space and Y represents a discrete set of sentiment classes. In the binary framework adopted for this study, the label set is restricted to $Y = \{0,1\}$, corresponding to positive and negative sentiments. The model is trained on a labelled dataset D = $\{(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)\}$ consisting of n text-label pairs, and optimized to minimize the classification error on unseen examples.

As illustrated in Figure 3, this study adopts the document-level formulation, assigning a single unified polarity, either positive or negative, to each entire IMDB movie review, rather than evaluating individual sentences or specific movie attributes.

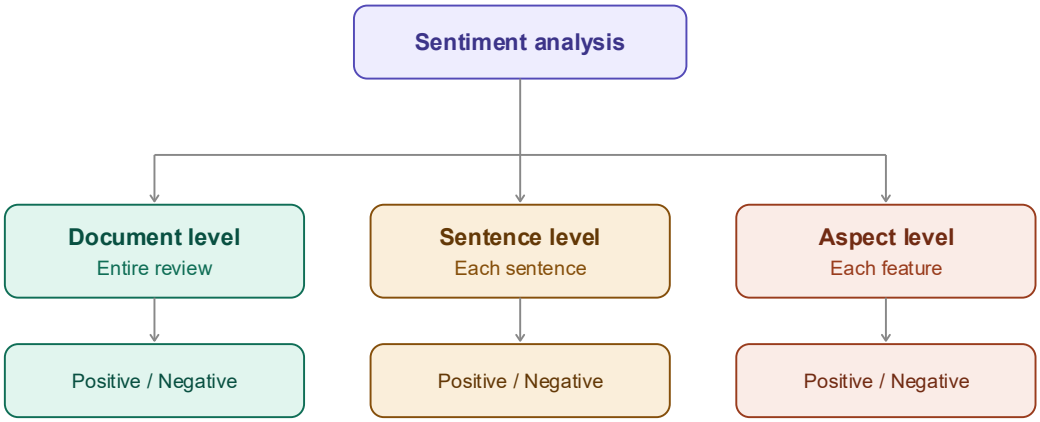


Figure 3. Levels of sentiment analysis granularity.

4.3 Neural Networks Fundamentals

The theoretical basis of modern NLP models like BERT lies in artificial neural networks, which are inspired by the structure and function of biological neurons in the human brain. While a biological neuron receives input signals through its dendrites, processes them in the cell body, and transmits an output signal through its axon when a certain activation threshold is exceeded, its artificial counterpart operates purely mathematically, where individual computational units, called neurons or nodes, receive numerical inputs, apply a weighted transformation, and produce an output through a non-linear activation function. Figure 4 presents the structure of an artificial neuron, illustrating the weighted input summation and the non-linear activation function applied to produce the output.

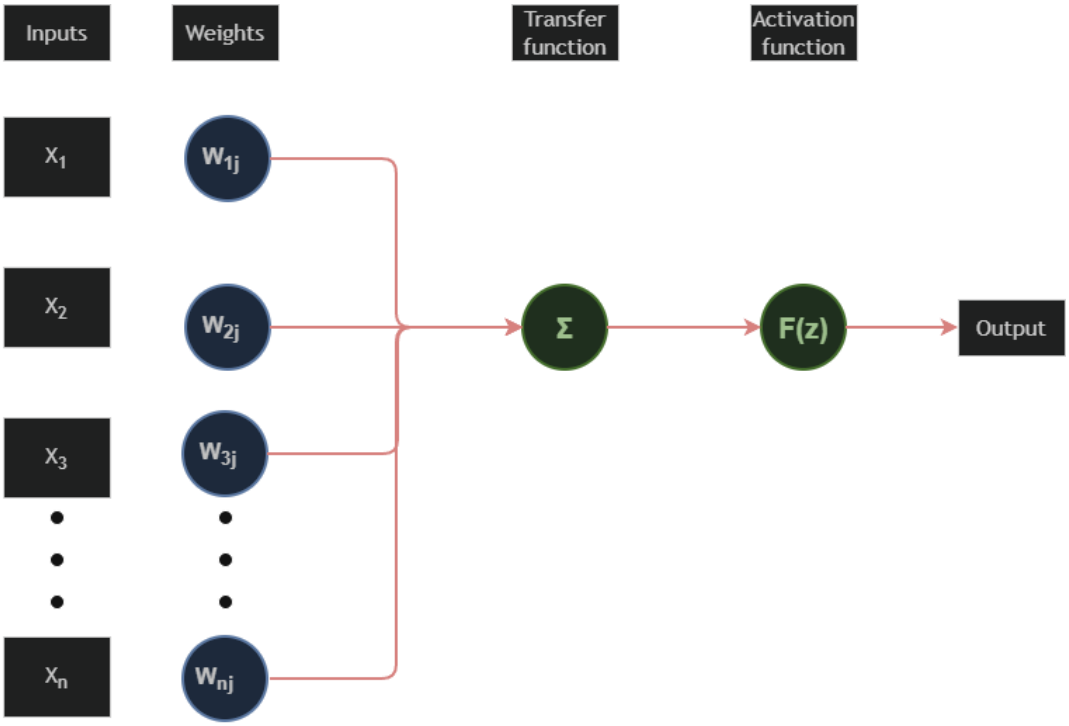


Figure 4. Structure of an artificial neuron.

4.3.1 The Artificial Neuron

The fundamental computational unit of a neural network is the artificial neuron, which computes a weighted sum of its inputs and passes the results through a non-linear activation function. Given an input vector $x = [x_1, x_2, ..., x_n]$, a weight vector $W = [w_1, w_2, ..., w_n]$, and a bias term b, the pre-activation output z is computed, as in (14):

$$z = \sum_{i=1}^{n} w_i x_i + b = W \cdot x + b \tag{14}$$

The result z is then passed through an activation function f(·), producing the neuron's output, as given in (15):

$$a = f(z) \tag{15}$$

Activation functions play a vital role in neural architecture. Without them, a multi-layer network would collapse into a single linear transformation, severely constraining its expressive

power. Non-linear activation functions enable neural networks to approximate arbitrarily complex functions, a property formalized by the Universal Approximation Theorem.

4.3.2 Activation Functions

Various activation functions have been proposed and adopted across different neural network architectures. The Sigmoid function maps any real-valued input to the range (0,1), making it popular for binary classification output layers, as given in (16):

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{16}$$

A major drawback of the sigmoid function is the vanishing gradient problem. When inputs reach large positive or negative values, the gradient saturates near zero, halting weight updates during backpropagation. The Rectified Linear Unit (ReLU) addresses this limitation by returning the input directly for positive values and zero otherwise, as shown in (17):

$$f(z) = \max(0, z) \tag{17}$$

Due to its computational efficiency and ability to preserve robust gradient flow during backpropagation, Relu has emerged as the default activation function for hidden layers in deep neural architectures. For multi-class and binary classification output layers, the Softmax function is used to convert a vector of raw output scores, known as logits, into a valid probability distribution over K classes, as in (18):

$$softmax(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, i = 1, \dots, K \tag{18}$$

In the present work, the BERT classification head produces two logits corresponding to the negative and positive classes, which are passed through a Softmax function to obtain class probabilities, with the predicted label assigned to the class with the highest probability.

4.3.3 Feedforward Neural Networks and Layers

Individual neurons are organized into layers to form a feedforward neural network, also referred to as Multi-Layer Perceptron (MLP). This standard architecture features an input layer for raw data ingestion, sequential hidden layers for non-linear feature extraction, and an output layer for the final prediction. The transformation applied by a single layer l can be expressed, as given in (19):

$$a^{(l)} = f\left(W^{(l)} a^{(l-1)} + b^{(l)}\right) \tag{19}$$

, where $W^{(l)}$ and $b^{(l)}$ are the learnable weight matrix and bias vector of layer l, and $a^{(l-1)}$ is the output of the preceding layer.

The depth of the network determines its capacity to model abstract and hierarchical representations of the input data.

4.3.4 Loss Function and Optimization

The training process of a neural network revolves around optimizing its parameters, the weight matrices W and bias vectors b, to minimize a loss function L that quantifies the discrepancy

between the model's predictions and the true labels. For both binary and multi-class classification frameworks, Cross-Entropy Loss serves as the standard objective function, defined as in (20):

$$L = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c}) \tag{20}$$

, where N represents the number of training examples, C is the number of classes, $y_{i,c}$ is the true binary indicator for class c of example i, and $\hat{y}_{i,c}$ is the predicted probability for that class.

In the binary case adopted in this work, the Cross-Entropy Loss simplifies to, as in (21):

$$L = -\frac{1}{N}\sum_{i=1}^{N}[y_i\log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)] \tag{21}$$

4.3.5 Backpropagation

The parameters of the network are updated progressively through an algorithm called backpropagation, which applies the chain rule of calculus to compute the gradient of the loss function with respect to each parameter in the network. For a parameter w in layer l, the gradient is computed, as in (22):

$$\frac{\partial L}{\partial w^{(l)}} = \frac{\partial L}{\partial a^{(l)}}\cdot\frac{\partial a^{(l)}}{\partial z^{(l)}}\cdot\frac{\partial z^{(l)}}{\partial w^{(l)}} \tag{22}$$

These gradients are then used to update the parameters in the direction that minimizes the loss, through an optimization algorithm such as Gradient Descent, illustrated in (23):

$$w \leftarrow w - \eta\cdot\frac{\partial L}{\partial w} \tag{23}$$

, where η represents the learning rate,a hyperparameter that controls the magnitude of each parameter update.

A large learning rate risks causing the optimization process to overshoot the minima and ultimately diverge. Consequently, selecting an optimal learning rate and designing an effective schedule are paramount for successful network optimization.

4.4 The Transformer Architecture

The Transformer architecture, introduced by Vaswani et al. in 2017 in the paper "Attention is All You Need", represented a fundamental departure from the recurrent and convolutional approaches that had previously dominated sequence modelling in NLP. In contrast to LSTM architectures, which process tokens sequentially, or CNN-based models, which rely on local convolutional windows, the Transformer architecture processes entire sequences simultaneously via the self-attention mechanism, which allows each token to directly attend to every other token in the sequence regardless of their relative distance [5].

Figure 5 shows the full Transformer architecture which consists of an encoder stack on the left and a decoder stack on the right. In the context of the present work, only the encoder component is relevant, as BERT is built exclusively on stacked Transformer encoders and does not employ a

decoder. Each encoder layer consists of two principal sub-components: a Multi-Head Self-Attention mechanism and a Position-wise Feed-Forward Network, with residual connections and layer normalization applied around each sub-component.

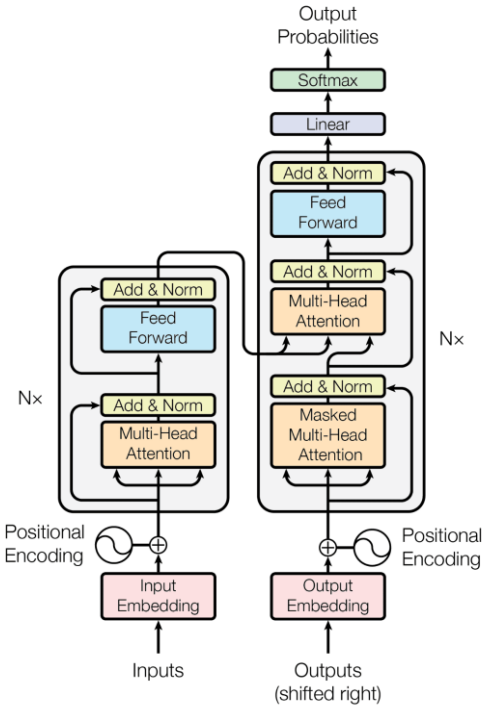


Figure 5. The Transformer architecture [12].

4.4.1 The Self-Attention Mechanism

At the heart of the Transformer lies the self-attention mechanism, designed to compute a dynamically weighted representation for each token based on its pairwise relevance to the entire sequence. The mechanism transforms the input into three distinct learnable linear projections: Query matrix (Q), the Key matrix (K), and the Value matrix (V). Given an input sequence represented as a matrix $X \in R^{nxd}$, where n is the sequence length and d is the model dimension, the three projections are computed, as given in (24)-(26):

$$Q = XW^Q \tag{24}$$
$$K = XW^K \tag{25}$$
$$V = XW^V \tag{26}$$

where $W^Q, W^K, W^V \in R^{dxdk}$ are learnable weight matrices.

Subsequently, the attention scores are computed via the dot product of the Query and Key matrices. To prevent these dot products from attaining excessively large magnitudes, which would cause the Softmax function to saturate into regions with vanishingly small gradients, the results are scaled by the square root of the key dimension d_k. This scaled dot-product attention is formulated, as presented in (27):

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{27}$$

The resulting attention weight matrix has dimensions R^{nxn}, where each entry a_{ij} represents the degree of attention that token i pays to token j when constructing its contextual representation. A high attention weight between two tokens indicates that the model has learned a strong contextual dependency between them, regardless of their positional distance in the sequence. This property is particularly important for sentiment analysis of long movie reviews, where sentiment-bearing expressions may be separated from the entities they describe by many intervening tokens.

4.4.2 Multi-Head Attention

Instead of relying on a single attention mechanism over the entire embedding dimension, the architecture utilizes Multi-Head Attention to compute h parallel attention functions. Each attention head processes a distinct, lower-dimensional linear projection of the queries, keys, and values. This parallelization enables the model to simultaneously synthesize information from various representational subspaces at different sequence positions, thereby capturing multiple types of contextual relationships simultaneously, as shown in (28):

$$MultiHead(Q,K,V) = Concat(head_1, head_2, \ldots., head_h)W^O \tag{28}$$

where each individual head is computed, as in (29):

$$head_1 = Attention\left(QW_i^Q, KW_i^K, VW_i^V\right) \tag{29}$$

and $W^O \in R^{hdvxd}$ is a learned output projection matrix.

4.4.3 Positional Encoding

Since the self-attention mechanism processes all tokens simultaneously and operates in a strictly permutation-invariant manner, meaning, it assigns the same representation regardless of sequence order-explicit positional information must be injected into the input representations. The original Transformer achieves this through Positional Encoding, which adds a fixed sinusoidal signal to each token embedding before it enters the encoder stack. For a token at position pos and embedding dimension index i, the positional encoding is defined, as in (30) and (31):

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right) \tag{30}$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right) \tag{31}$$

The use of sinusoidal functions of different frequencies ensures that each position receives a unique encoding, and that the model can generalize to sequence lengths not seen during training. BERT replaces fixed sinusoidal positional encodings with learned positional embeddings, which are trained jointly with the rest of the model parameters during pre-training. As illustrated in Figure 6, the sinusoidal positional encoding assigns a unique pattern of values to each position across all embedding dimensions, ensuring that no two positions share identical encodings.

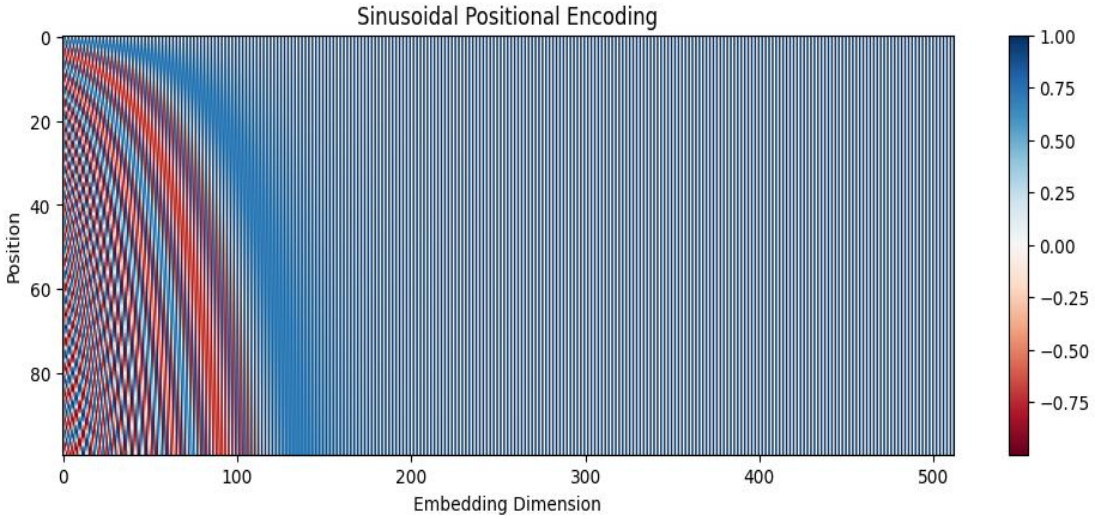


Figure 6. Sinusoidal positional encoding matrix.

4.4.4 Position-wise Feed-Forward Network

Following the multi-head attention sub-layer, each Transformer encoder layer applies an identical Position-wise Feed-Forward Network (FFN) independently to each token position. This consists of two linear transformations with a ReLU activation in between, as given in (32):

$$FFN(x) = \max(0, xW_1 + b_1)\, W_2 + b_2 \tag{32}$$

, where $W_1 \in R^{dxd}_ff$ and $W_2 \in R^{d}_ff^{xd}$ are learnable weight matrices.

In BERT-Base, the feed-forward intermediate dimension is *d_ff = 3072*, four times the model dimension of *d = 768*. The feed-forward sub-layer introduces additional non-linear transformation capacity beyond what the attention mechanism provides, enabling each token's representation to be further refined after contextual information has been aggregated through attention.

4.4.5 Residual Connections and Layer Normalization

Both the multi-head attention and feed-forward sub-layers in the encoder block feature residual connections and Layer Normalization. The residual shortcut routes the sub-layer's initial input to be added directly to its output prior to normalization, operating as follows in (33):

$$output = LayerNorm\big(x + Sublayer(x)\big) \tag{33}$$

Residual connections address the vanishing gradient problem in deep networks by providing a direct gradient pathway from the output back to earlier layers, enabling the training of very deep architectures without degradation. Layer Normalization stabilizes the activations across the hidden dimension at each layer, accelerating convergence and improving training stability. In BERT-Base, these mechanisms are applied across all 12 stacked encoder layers,enabling the model to build token representations with depth.

4.5 BERT Architecture

BERT (Bidirectional Encoder Representations from Transformers) was introduced by Devlin et al.at Google AI in 2019 and rapidly established itself as the dominant pre-trained language model for a wide range of NLP tasks [5]. Unlike its predecessors, which either processed text unidirectionally or combined two separate unidirectional models, BERT leverages the full Transformer encoder stack to learn deeply bidirectional representations of text, conditioning each token's representation on both its left and right context simultaneously across all layers.

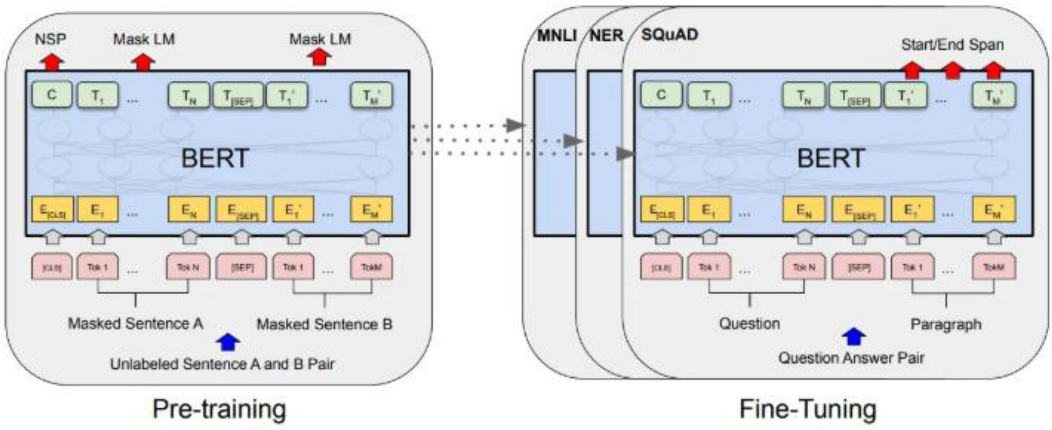


Figure 7. BERT input representation and pre-training objectives [5].

4.5.1 Model Architecture

As illustrated in Figure 7, BERT is built exclusively upon stacked Transformer encoder blocks, as detailed in Section 3.2, BERT was introduced in two primary configurations that vary in depth, width, and computational overhead. The base configuration, BERT-Base, employs a moderate number of encoder layers and attention heads, resulting in approximately 110 million trainable parameters. This scale is enough to capture rich contextual representations across a wide range of NLP tasks while remaining computationally tractable for fine-tuning on standard hardware. Conversely, BERT-Large significantly expands the network's dimensions, encompassing roughly 340 million parameters to deliver better performance gains at the cost of substantial memory and training time requirements [5]. Both configurations share the same pre-training methodology and fine-tuning protocol, differing only in model capacity. Ultimately, the architectural depth of both configurations allows BERT to construct highly abstract, contextually enriched token representations with each successive encoder layer, a property that fundamentally distinguishes it from shallower models such as LSTM or CNN architectures.

4.5.2 Input Representation

BERT constructs its input representation by combining three distinct learned embedding vectors for each input token, Given a token at position i, its input embedding E_i is defined as in (34):

$$E_i = TokenEmbedding(t_i) + SegmentEmbedding(s_i) + PositionEmbedding(i) \quad (34)$$

The Token Embedding maps each subword token to a dense vector from a learned vocabulary constructed using the WordPiece subword tokenization algorithm. WordPiece

decomposes rare or unseen words into subword units, ensuring that the model can represent any input text without encountering out-of-vocabulary tokens [5].

The Segment Embedding distinguishes between two input sentences in tasks that require sentence-pair reasoning, assigning a distinct embedding to tokens belonging to each sentence respectively. For single-sequence tasks such as sentiment classification, all tokens receive the same segment embedding.

The Position Embedding encodes the absolute position of each token within the sequence, supporting sequences of up to 512 tokens. Unlike the fixed sinusoidal positional encodings used in the original Transformer, BERT's positional embeddings are learned jointly with all other model parameters during pre-training, allowing the model to adaptively weight positional information based on the training signal.

Two special tokens are prepended and appended to every input sequence. The [CLS] token is inserted at the beginning of the sequence, and its final hidden state serves as the aggregate sequence representation that can be used for classification tasks. The [SEP] token marks the boundary between two input sentences in sentence-pair tasks, and terminates the sequence in single-sentence tasks.

4.5.3 Pre-Training

BERT is pre-trained on two complementary self-supervised objectives using a combined corpus of approximately 3.3 billion words from English Wikipedia and the BooksCorpus [5].

One of the two objectives is called Masked Language Modeling (MLM). MLM randomly selects 15% of the input tokens in each sequence for masking. Of the selected tokens, 80% are replaced with the special [MASK] token, 10% are replaced with a random token from the vocabulary, and 10% are left unchanged.The model is then trained to predict the original token at each masked position from its surrounding bidirectional context. Formally, for a masked token at position i with true token x_i, the MLM loss is expressed as in (35):

$$L_{MLM} = -logP(x_i|\tilde{x}) \tag{35}$$

, where $\tilde{x}$ denotes the masked input sequence.

The use of the mixed substitution strategy prevents the model from learning to expect a [MASK] token at fine-tuning time where no masking occurs, thereby reducing the mismatch between pre-training and fine-tuning conditions.

The other objective is called Next Sentence Prediction (NSP) which trains the model to determine whether a given pair of sentences appears consecutively in the original corpus. Half of the training pairs are genuine consecutive sentences and half are randomly sampled non-consecutive pairs.The [CLS] token representation is used to predict the binary label, enabling the model to capture inter-sentence coherence and discourse-level relationships. The combined pre-training loss is defined in (36):

$$L_{pre-train} = L_{MLM} + L_{NSP} \tag{36}$$

4.5.4 Fine-Tuning for Sentiment Classification

Fine-tuning adapts the pre-trained BERT model to a specific downstream task by adding a minimal task-specific output layer and training the entire model end-to-end on labelled data. For binary sentiment classification, a linear classification head is applied to the [CLS] token representation $h_{\{[CLS]\}}$ produced by the final encoder layer, as shown in (37):

$$\hat{y} = softmax\big(W \cdot h_{[CLS]} + b\big) \tag{37}$$

, where W and b are the learnable parameters of the classification head, and the output $\hat{y}$ is a probability distribution over the two sentiment classes.

The predicted label is assigned, as in (38):

$$\hat{c} = \arg\max \hat{y}_c \tag{38}$$

The model is trained by minimizing the Cross-Entropy Loss between the predicted probabilities and the true labels, as defined in (39):

$$L_{fine-tune} = -\sum_{c=0}^{1} y_c \log(\hat{y}_c) \tag{39}$$

During fine-tuning, all parameters of BERT are updated jointly on the labelled training set. This end-to-end fine-tuning strategy, as opposed to using BERT purely as a fixed feature extractor, has been consistently demonstrated to yield superior performance across classification tasks [5]. The relatively small number of additional parameters introduced by the classification head means that the dominant computational cost of fine-tuning lies in updating the pre-trained encoder weights, which requires careful optimization to avoid forgetting of the representations learned during pre-training.

4.6 Evaluation Metrics

The evaluation of a sentiment classification model relies on a set of standard metrics derived from the confusion matrix. The confusion matrix presented in Figure 8 is a table that summarises the predictions made by the model on the test set, organising all outcomes into four categories:

- True Positives (TP), cases where the model correctly predicted positive
- True Negatives (TN), cases where the model correctly predicted negative
- False Positives (FP), cases where the model incorrectly predicted positive for a negative review
- False Negatives (FN), cases where the model incorrectly predicted negative for a positive review.

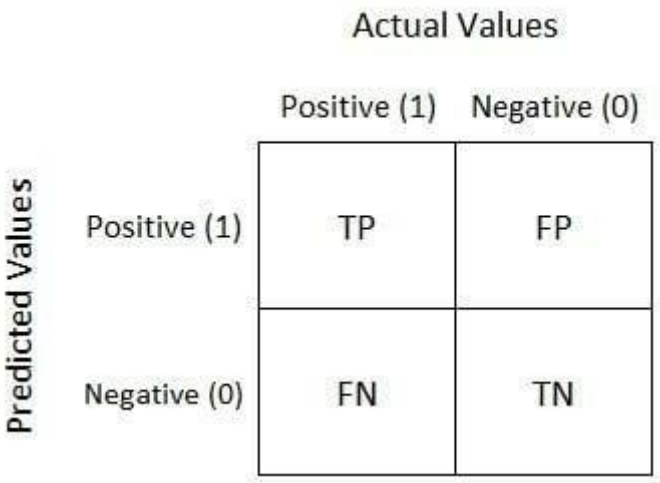


Figure 8. Confusion matrix

The performance of a sentiment classification model is evaluated through four standard metrics derived from the confusion matrix, produced by the model on a held-out test set. Accuracy measures the overall fraction of correctly classified instances,being defined in (40):

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{40}$$

Precision measures the fraction of predicted positive instances that are actually positive, reflecting the model's ability to minimize false positives, as expressed in (41):

$$Precision = \frac{TP}{TP + FP} \tag{41}$$

Recall measures the fraction of actual positive instances that are correctly identified, reflecting the model's ability to detect all positive cases, as given in (42):

$$Recall = \frac{TP}{TP + FN} \tag{42}$$

The F1 score is the harmonic mean of Precision and Recall, providing a balance measure of performance particularly useful when the two metrics diverge, as presented in (43):

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \tag{43}$$

These four metrics collectively provide a comprehensive picture of model performance, capturing not only overall correctness but also the trade-off between false positives and false negatives.

The F1 score is especially important in classification tasks because accuracy alone can be misleading. For example, a model that always predicts the majority class can achieve high accuracy on an imbalanced dataset while completely failing on the minority class. The F1 score avoids this by equally weighting precision and recall, making it a more reliable indicator of real model performance. In this work, since the IMDB dataset is balanced between positive and negative reviews, accuracy and F1 score are expected to produce similar values, but both are reported to give a complete picture of how the model performs.

5. Implementation

5.1 Technologies and Tools

The implementation of this project was carried out in Python 3.10.20, managed through an Anaconda virtual environment**,** which ensured full isolation of dependencies and reproducibility of the execution environment across different machines. The training pipeline was executed on Google Colab using an NVIDIA T4 GPU, which provided the computational resources necessary for fine-tuning BERT. Preprocessing, local development, and the web application were developed and tested in Visual Studio Code on a Windows machine, as shown in Figure 9:

```
import torch
from transformers import BertTokenizer, BertForSequenceClassification
from datasets import load_dataset, Dataset, DatasetDict, load_from_disk
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import gradio as gr
```

Figure 9: Core library imports used throughout the project

The torch library served as the core learning framework, providing the tensor computation engine, automatic differentiation and GPU acceleration underlying the entire pipeline. All model operations were performed using PyTorch primitives.

The transformers library provided the interface for loading the pre-trained bert-base-uncased model and BertTokenizer. The BertForSequenceClassification class was used to instantiate BERT with a classification head appended on top of the encoder, enabling end-to-end fine-tuning for binary sentiment classification.

The datasets library was used to retrieve the IMDB dataset and provide data structures for large-scale text data, supporting batched tokenization and disk-based caching to avoid repeated tokenization across training runs.

The scikit-learn library contributed to two functionalities: the train_test_split function which was used to partition the dataset into training, validation, and test subsets, while accuracy_score and precision_recall_fscore_support were used to compute the final evaluation metrics on the test set.

The pandas and numpy libraries supported tabular data manipulation during the preprocessing phase, handling DataFrame operations such as concatenation, deduplication, and label encoding.The matplotlib library was used to generate all visualization outputs, including the training history curves, the MAX_LENGTH comparison charts, the confusion matrix, and the review length distribution histogram.

Finally, the gradio library was used to develop the interactive web interface for the sentiment analysis application.

5.2 Training and Evaluation branch

The sentiment analysis system developed in this work consists of two interconnected pipelines that share a common preprocessing and tokenization foundation: a training and evaluation branch, responsible for fine-tuning the BERT model and evaluating its performance, and an inference branch, responsible for processing new reviews through the deployed web application. The overall architecture of the system is illustrated in Figure 10.

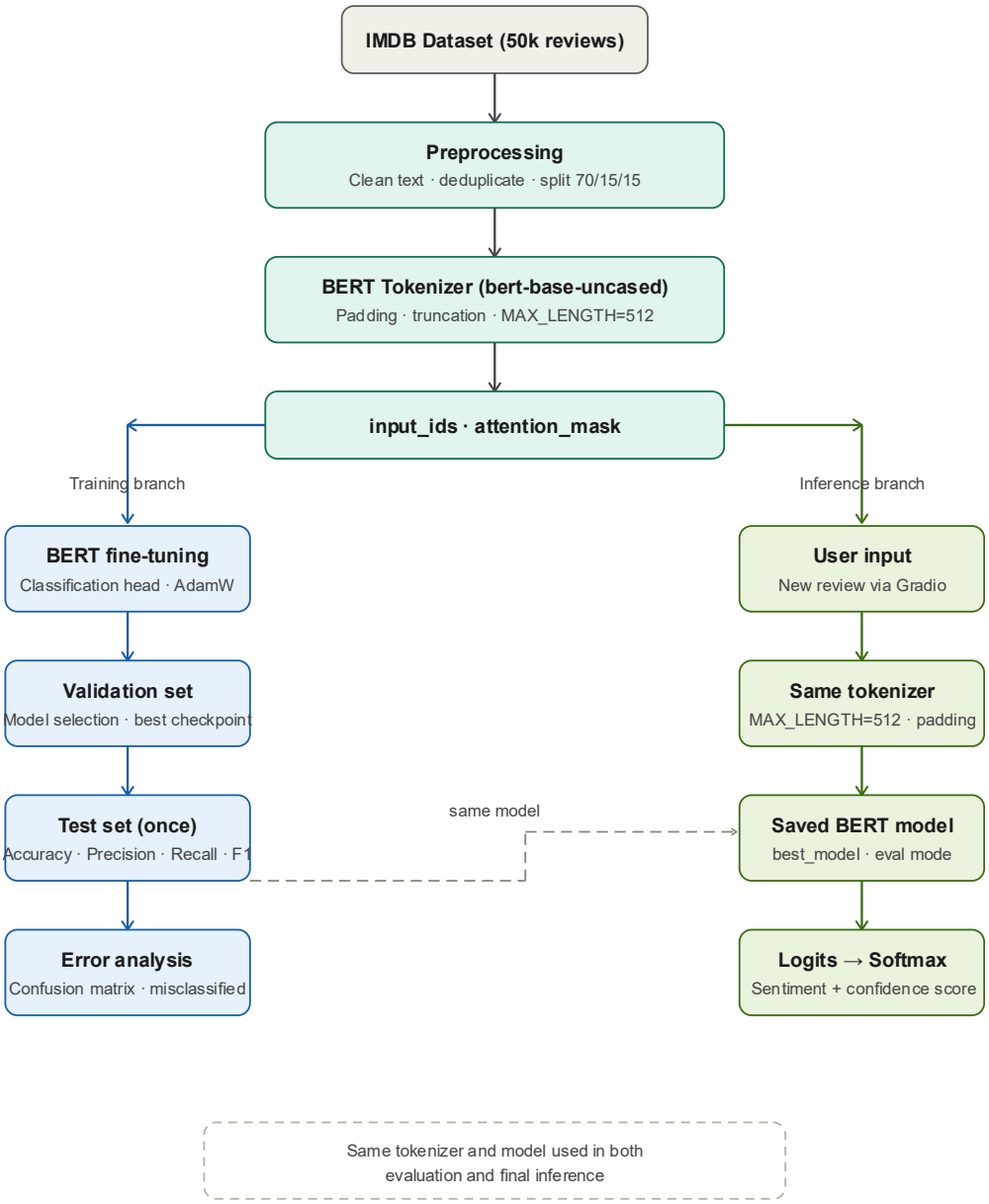


Figure 10. Training and Evaluation branch

The branch begins with the loading of the raw IMDB dataset, which is passed through a preprocessing stage comprising HTML tag removal, whitespace normalisation, duplicate elimination, and stratified splitting into training, validation, and test subsets. The preprocessed text is then tokenized using the BertTokenizer associated with the bert-base-uncased checkpoint, with padding and truncation applied to a uniform maximum length of 512 tokens. This tokenization step produces two tensors for each input sequence: input_ids, which encodes the token indices, and attention_mask, which distinguishes real tokens from padding positions.

At this point the branches diverges into two parts.

5.2.1 Training Branch

In the training branch, the tokenized data is used to fine-tune a BertForSequenceClassification model with a binary classification head appended to the [CLS] token representation. The model is optimised using the AdamW algorithm with a linear learning rate schedule with warm-up. At the end of each training epoch, the model is evaluated on the validation set, and the checkpoint producing the highest validation accuracy is saved to disk. Once training is complete, the saved model is evaluated exactly once on the held-out test set to produce the final performance metrics: accuracy, precision, recall, and F1 score. An error analysis is subsequently performed on the misclassified examples to identify systematic patterns in the model's errors.

5.2.2 Inference Branch

In the inference branch, the same BertTokenizer and the same saved model are used to process new text inputs submitted by the user through the Gradio web interface. When a user submits a review, the text is tokenized using the identical configuration employed during training, the same bert-base-uncased tokenizer, the same MAX_LENGTH=512, and the same padding and truncation settings, producing input_ids and attention_mask tensors that are passed directly to the saved model. The model runs in evaluation mode, which disables dropout, and produces logit scores for the two classes. A softmax function is applied to the logits to obtain class probabilities, from which the predicted sentiment label and the associated confidence score are derived and returned to the user.

A key design principle of this system is the strict consistency between training and inference: the tokenizer, the maximum sequence length, and the model weights used during evaluation are identical to those used in the deployed application. This ensures that the performance measured on the test set is representative of the performance experienced by end users of the application.

5.3 Dataset and Preprocessing

5.3.1 Dataset

The dataset used in this work is the IMDB Large Movie Review Dataset, introduced by Maas et al. (2011) [16]. As illustrated in Figure 11, the dataset contains 49,582 unique reviews, containing an equal number of reviews for each class, collected from the Internet Movie Database and added with binary sentiment labels, positive or negative. It has become one of the most adopted benchmarks for binary sentiment classification, providing a standardized evaluation surface for comparing models across methodological generations.

Implementation began by assessing an Exploratory Data Analysis (EDA) performed to verify the data integrity, revealing a dataset with no missing or empty text fields. A character-level length analysis revealed a median of 957, a mean of 1,289, and a maximum of 13,593 characters. This highlighted that a significant portion of the reviews exceeds BERT 512-token constraint, requiring truncation during preprocessing. Additionally, the EDA flagged duplicate rows and embedded HTML tags (such as
), which were resolved via a targeted cleaning and deduplication stage.

review	sentiment
49582 unique values	2 unique values
One of the other reviewers has mentioned that after watching just 1 Oz episode you'll be hooked. The...	positive
A wonderful little production. The filming technique is very unassuming- very old-time-B...	positive
I thought this was a wonderful way to spend time on a too hot summer weekend, sitting in the air con...	positive
Basically there's a family where a little boy (Jake) thinks there's a zombie in his closet & his par...	negative
Petter Mattei's "Love in the Time of Money" is a visually stunning film to watch. Mr. Mattei offers ...	positive
Probably my all-time favorite movie, a story of selflessness, sacrifice and dedication to a noble ca...	positive

Figure 11. IMDB Dataset

5.3.2 Preprocessing

Building upon the insights gained from the Exploratory Data Analysis (EDA) phase, a preprocessing pipeline was implemented to prepare the textual data for the Transformer model. From Figure 12, it can observed how in the first step, the raw dataset was loaded using the datasets library, and its default train and test splits were combined into a single unified DataFrame. This concatenation was necessary in order to apply the preprocessing steps, including HTML tag removal and deduplication across the entire dataset, before performing the custom train/validation/test split, ensuring that no duplicate entries appear across different subsets.

```
raw = load_dataset("imdb")
train_df = pd.DataFrame(raw["train"])
test_df  = pd.DataFrame(raw["test"])
df = pd.concat([train_df, test_df], ignore_index=True)
```

Figure 12: Dataset loading and split combination

In the second step, the raw text was cleaned by removing HTML tags using a regular expression pattern and normalizing whitespace by collapsing consecutive whitespace characters into a single space, as illustrated in Figure 13. Text cleaning was necessary because IMDB reviews contain HTML formatting artifacts that carry no semantic content relevant to sentiment classification, and inconsistent whitespace that could interfere with the tokenization process. This was immediately followed by a deduplication step to prevent data leakage and overfitting. After duplicate removal, the dataset contained 49,578 unique reviews.

```
def clean_text(text):
    text = re.sub(r"<.*?>", " ", text)       # remove HTML tags
    text = re.sub(r"\s+", " ", text).strip() # collapse whitespace
    return text
```

Figure 13: Text cleaning function

In the third step, the sanitized dataset was then divided into three non-overlapping subsets using a stratified two-stage split strategy. Stratification was applied at each stage to preserve the original class distribution across all three partitions. The adopted ratios were training (70%), validation (15%), and testing (15%), yielding 34,704 training samples, 7,437 validation samples, and 7,437 test samples. A fixed random seed of 42 was applied throughout to ensure full reproducibility across independent runs, as shown in Figure 14:

```
train_val_df, test_df = train_test_split(
    df, test_size=0.15, random_state=42, stratify=df["label"]
)
relative_val_size = 0.15 / (1.0 - 0.15)  # ≈ 0.176
train_df, val_df = train_test_split(
    train_val_df, test_size=relative_val_size,
    random_state=42, stratify=train_val_df["label"]
)
```

Figure 14: Stratified two-stage dataset split

In the fourth step, text tokenization was executed using the Wordpiece tokenizer provided by bert-base-uncased. The uncased variant was selected because movie review sentiment is not meaningfully dependent on token capitalisation, allowing for a more compact and efficient vocabulary. Each review was tokenized with the addition of the special tokens required by BERT, a [CLS] token at the sequence start and a [SEP] token at the end. Sequences exceeding the maximum token length were truncated, while shorter sequences were padded to a uniform length to enable batched processing, as is expressed in Figure 15. Padding ensures tensors of equal length within each batch, while truncation limits sequences exceeding 512 tokens.

```
def tokenize_fn(batch):
    return tokenizer(
        batch["text"],
        truncation=True,
```

```
        padding="max_length",
        max_length=MAX_LENGTH,
    )
tokenized = dataset_dict.map(tokenize_fn, batched=True, batch_size=500)
```

Figure 15: Tokenization function using BertTokenizer

Post-tokenization, each data sample is transformed into three specific tensors configured for the Transformer encoder: input_ids for vocabulary mapping, an attention_mask to distinguish real tokens from padding positions, and token_type_ids, which identifies sentence boundaries. A sanity check confirmed that all three tensors were correctly shaped at [512] per sample, with labels correctly encoded as integer values.

In the fifth and final step, the tokenized dataset was saved to disk in the DatasetDict format, avoiding redundant recomputation of the tokenization pipeline across training runs.

5.4 Model Configuration

The model selected for this task is bert-base-uncased, a pre-trained transformer encoder comprising 12 attention layers, 12 attention heads, a hidden dimensionality of 768, and approximately 110 million trainable parameters. The base variant was preferred over bert-large on the idea that, the large variant doubles the number of layers and hidden dimensions, substantially increasing memory requirements and training time without guaranteeing better performance gains on a well-defined binary classification task. The uncased variant was selected over its cased counterpart because sentiment polarity in movie reviews is not systematically encoded through capitalisation, and the uncased vocabulary reduces tokenization complexity without loss of relevant information.

For the downstream classification task, the pre-trained BERT encoder was extended with a specific classification head by instantiating BertForSequenceClassification with num_labels=2, as shown in Figure 16. This adds a linear projection layer on top of the [CLS] token representation, mapping the 768-dimensional hidden state to a 2-dimensional output space over which a softmax distribution is computed during inference:

```
model = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2,
)
model = model.to(DEVICE)
```

Figure 16: BERT model instantiation with classification head

When loading the pre-trained weights, the framework reports two categories of parameter status. Parameters marked as UNEXPECTED correspond to the masked language modelling and next sentence prediction heads present in the original pre-trained checkpoint, which are not required for sequence classification and are safely discarded. Parameters marked as MISSING correspond to the newly added classification head, whose weights are randomly initialised and subsequently learned during fine-tuning. A sanity check was performed on the loaded model by running a forward pass on a small batch from the training set, yielding an initial loss of approximately 0.69, consistent with the theoretical value of $\ln(2)$ expected from an untrained binary classifier producing uniform output probabilities.

The optimiser used throughout training is AdamW, as defined in Figure 17, with a learning rate of 2×10^{-5} and a weight decay coefficient of 0.01 applied to all non-bias parameters:

```
optimizer = AdamW(
    model.parameters(),
    lr=2e-5,
    weight_decay=0.01
)
```

Figure 17: AdamW optimiser configuration

A linear learning rate schedule with warm-up was employed, as expressed in Figure 18. During the warm-up phase, comprising the first 10% of total training steps, the learning rate was linearly increased from zero to its target value, after which it decayed linearly to zero over the remaining steps:

```
total_steps = len(train_loader) * NUM_EPOCHS
warmup_steps = int(0.1 * total_steps)
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=warmup_steps,
    num_training_steps=total_steps
)
```

Figure 18: Linear learning rate schedule with warm-up

Gradient clipping was applied at each training step with a maximum gradient norm of 1.0, preventing large gradient updates that could destabilise training. The batch size was set to 16 and training was conducted for 3 epochs. The complete hyperparameter configuration used in this work is presented in Table 5.

Table 5: Hyperparameter configuration used for BERT fine-tuning

Hyperparameter	Value
Model	bert-base-uncased
MAX_LENGTH	512
Batch size	16
Epochs	3
Optimiser	AdamW
Learning rate	2×10^{-5}
Weight decay	0.01
Warm-up steps	10% of total steps
Gradient clipping	1.0
Random seed	42

5.5 Training

The training pipeline was executed on Google Colab using an NVIDIA T4 GPU, which provided the computational resources necessary for fine-tuning BERT within a reasonable time frame. Before starting the training loop,the tokenized dataset was loaded from disk and wrapped into DataLoader objects for each split. The training DataLoader used shuffling to ensure a different batch order at each epoch, while the validation and test DataLoaders kept a fixed order,as shown in Figure 19:

```
train_loader = DataLoader(
    dataset["train"],
    batch_size=BATCH_SIZE,
    shuffle=True,
)
val_loader = DataLoader(
    dataset["val"],
    batch_size=BATCH_SIZE,
    shuffle=False,
)
```

Figure 19: DataLoader configuration for training and validation splits

A sanity check was performed before training to verify that the DataLoaders were producing correctly shaped batches. A single batch was loaded and its dimensions were inspected, confirming that input_ids and attention_mask tensors had the shape [16, 512] and that labels had the shape [16], consistent with the expected batch size and sequence length.

The training loop ran for 3 epochs. At each epoch, the model was set to training mode and all batches from the training DataLoader were processed sequentially. For each batch, the input tensors were moved to the GPU. A forward pass was performed to compute the loss, and backpropagation was applied to compute the gradients. Gradient clipping was applied before the optimiser step, and the learning rate scheduler was updated after each batch, as shown in Figure 20:

```
for epoch in range(NUM_EPOCHS):
    model.train()
    for batch in train_loader:
        optimizer.zero_grad()
        outputs = model(
            input_ids=batch["input_ids"].to(DEVICE),
            attention_mask=batch["attention_mask"].to(DEVICE),
            labels=batch["labels"].to(DEVICE)
        )
        loss = outputs.loss
```

```
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        scheduler.step()
```

Figure 20: Training loop

At the end of each epoch, the model was evaluated on the validation set. The model was set to evaluation mode, which disables dropout, and predictions were collected without computing gradients using torch.no_grad(). Validation accuracy was computed and compared against the best result seen so far. If the current epoch produced a higher validation accuracy, the model weights were saved to disk as the best checkpoint, as shown in Figure 21:

```
model.eval()
all_preds, all_labels = [], []

with torch.no_grad():
    for batch in val_loader:
        outputs = model(
            input_ids=batch["input_ids"].to(DEVICE),
            attention_mask=batch["attention_mask"].to(DEVICE)
        )
        preds = torch.argmax(outputs.logits, dim=1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(batch["labels"].numpy())

val_acc = accuracy_score(all_labels, all_preds)

if val_acc > best_val_acc:
    best_val_acc = val_acc
    model.save_pretrained("best_model")
```

Figure 21: Validation and best model saving

5.6 MAX_LENGTH Comparison experiment

Since BERT supports a maximum sequence length of 512 tokens, but many fine-tuning studies use shorter lengths to reduce training time, a dedicated experiment was conducted to determine the optimal value for this parameter. Three values were evaluated: 128, 256, and 512 tokens. For each configuration, the model was fine-tuned from scratch for 3 epochs using the same hyperparameters described in Section 5.3, and performance was measured exclusively on the validation set to avoid any influence on the final test evaluation.

For each MAX_LENGTH value, a separate training and evaluation function was defined that tokenizes the dataset using the specified maximum sequence length, initializes a new model instance, and runs the full training loop, returning the final validation accuracy, F1 score, and total training time, as shown in Figure 22:

```
def train_and_evaluate(max_length):
    tokenized = prepare_data(max_length)
    model = BertForSequenceClassification.from_pretrained(
        "bert-base-uncased", num_labels=2
    ).to(DEVICE)
    optimizer = AdamW(model.parameters(), lr=2e-5, weight_decay=0.01)
    total_steps = len(train_loader) * NUM_EPOCHS
    scheduler = get_linear_schedule_with_warmup(
        optimizer,
        num_warmup_steps=int(0.1 * total_steps),
        num_training_steps=total_steps
    )
    for epoch in range(NUM_EPOCHS):
        train_one_epoch(model, train_loader, optimizer, scheduler)
        val_acc, val_f1 = evaluate(model, val_loader)
    return val_acc, val_f1
```

Figure 22: Training and evaluation function used in the MAX_LENGTH comparison experiment

5.7 Evaluation

After training and model selection were completed, the final model was evaluated on the test set. The test set was used once, after all hyperparameter decisions had been made based on the validation set performance. The evaluation was performed using the four metrics defined in Section 4.3, and the results are presented in Table 4.

The evaluation was done by running the model in evaluation mode over all batches in the test DataLoader, collecting predictions and true labels, and computing the final metrics using accuracy_score and precision_recall_fscore_support from the scikit-learn library, as shown in Figure 23:

```
model.eval()
all_preds, all_labels = [], []
with torch.no_grad():
    for batch in test_loader:
        outputs = model(
```

```
            input_ids=batch["input_ids"].to(DEVICE),
            attention_mask=batch["attention_mask"].to(DEVICE)
        )
        preds = torch.argmax(outputs.logits, dim=1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(batch["labels"].numpy())
accuracy = accuracy_score(all_labels, all_preds)
precision, recall, f1, _ = precision_recall_fscore_support(
    all_labels, all_preds, average="binary"
)
```

Figure 23: Test set evaluation and metric computation

To provide a visual breakdown of the prediction outcomes across both classes, a confusion matrix was generated using matplotlib and seaborn, as shown in Figure 24:

```
cm = confusion_matrix(all_labels, all_preds)


sns.heatmap(
    cm, annot=True, fmt="d", cmap="Blues",
    xticklabels=["Negative", "Positive"],
    yticklabels=["Negative", "Positive"]
)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix — BERT on IMDB Test Set")
plt.savefig("confusion_matrix.png")
```

Figure 24: Confusion matrix generation

The resulting confusion matrix is shown in Figure 25, displaying the number of correct and incorrect predictions for each class across the full test set of 7,437 samples.

Additionally, an error analysis was performed on the misclassified examples by extracting the original review texts from test_texts.csv, which was saved during the preprocessing phase to preserve the exact text corresponding to each test sample. This allowed for a direct inspection of the reviews that the model classified incorrectly, enabling the identification of patterns in the errors.

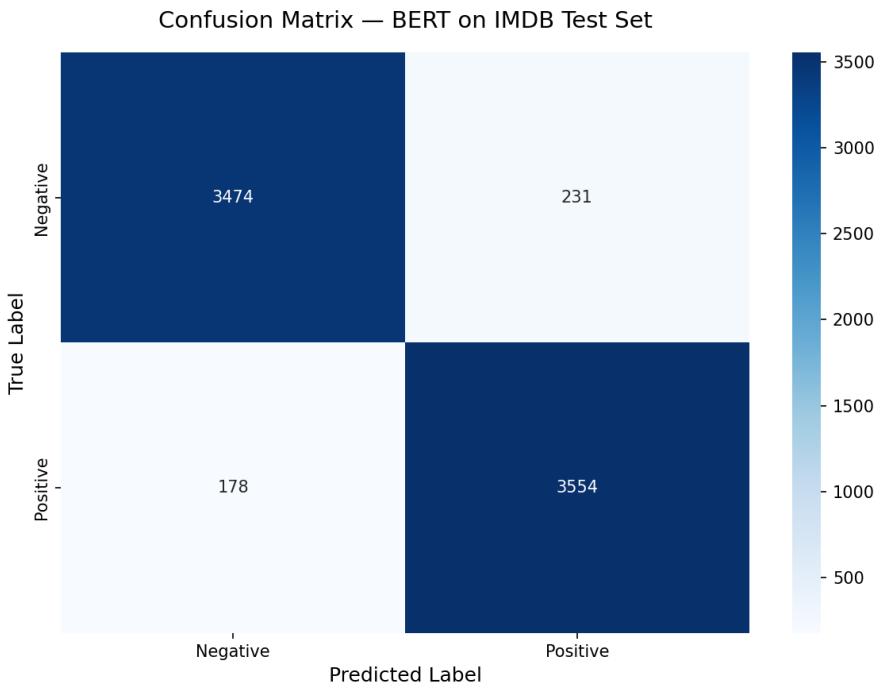


Figure 25: Confusion Matrix BERT on IMDB Test Set

5.8 User Interface

After training and evaluation were completed, the fine-tuned model was integrated into an interactive web application using the gradio library. The application allows users to enter a review and receive a sentiment prediction, positive or negative, along with a confidence score representing the probability assigned by the model to the predicted class.

The prediction function takes a raw text string as input, tokenizes it using the same tokenizer configuration employed during training, runs a forward pass through the model, and applies the softmax function to the output logits to obtain class probabilities, as shown in Figure 26:

```
def predict(text):
    inputs = tokenizer(
        text,
        truncation=True,
        padding="max_length",
        max_length=MAX_LENGTH,
        return_tensors="pt"
    )
    with torch.no_grad():
        outputs = model(
            input_ids=inputs["input_ids"].to(DEVICE),
            attention_mask=inputs["attention_mask"].to(DEVICE)
        )
```

```
    probs = torch.softmax(outputs.logits, dim=1)[0]
    pred_label = torch.argmax(probs).item()
    confidence = probs[pred_label].item()
    label = "POSITIVE " if pred_label == 1 else "NEGATIVE "
    return label, f"{confidence * 100:.2f}%"
```

Figure 26: Prediction function used in the Gradio interface

The Gradio interface,as illustrated in Figure 27, was configured with a text input box, two output fields for the sentiment label and confidence score, and a set of example inputs to allow users to quickly test the model without typing. The application was initially developed and tested locally, after which it was deployed to Hugging Face Spaces, a free platform for hosting machine learning demos that runs independently of any local machine, making the application available 24/7 through a public URL.

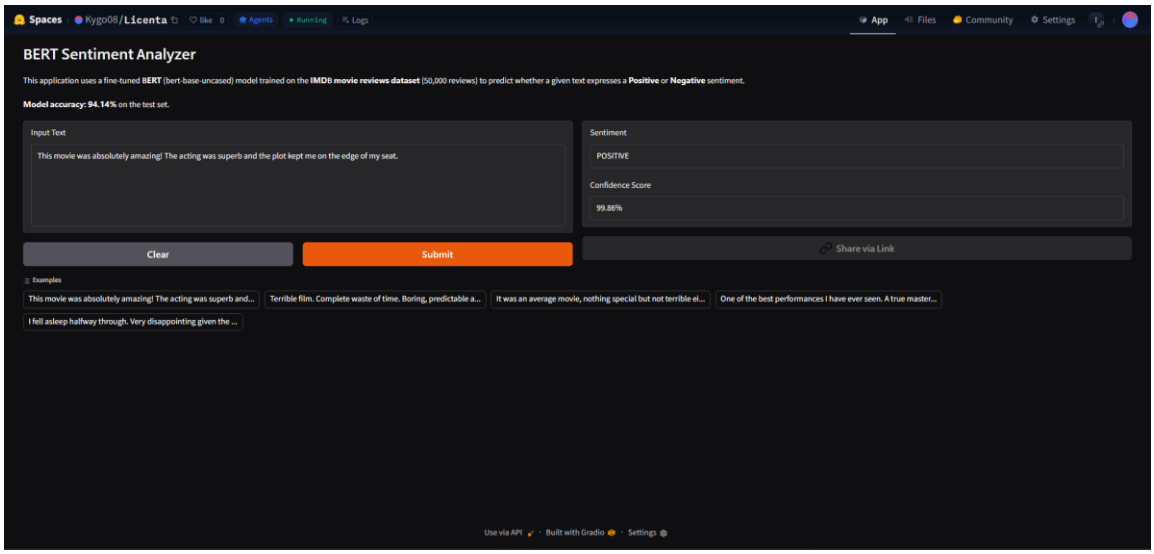


Figure 27. Gradio interface

6. Experimental Results

6.1 Training Results

The fine-tuned BERT model was trained for 3 epochs on the training set of 34,704 samples, with validation performed at the end of each epoch. The training results are presented in Table 6. Accuracy and loss are both reported because they capture different aspects of model performance. Accuracy measures prediction correctness, while loss reflects prediction confidence and optimization quality, making the two metrics complementary.

Table 6: Training results per epoch

Epoch	Train Loss	Train Accuracy	Val Loss	Val Accuracy
1	0.2771	88.35%	0.1652	93.79%
2	0.1425	95.84%	0.2563	93.69%
3	0.0746	98.17%	0.2540	94.62%

Several observations can be made from these results. Training loss decreased consistently across all three epochs, dropping from 0.2771 at epoch 1 to 0.0746 at epoch 3, which indicates that the model learned effectively from the training data. Training accuracy followed the same pattern, reaching 98.17% by the final epoch.

Validation accuracy, on the other hand, behaved differently. It started at 93.79% at epoch 1, slightly decreased to 93.69% at epoch 2, and then improved to 94.62% at epoch 3. The small drop at epoch 2 followed by a recovery at epoch 3 is a normal variation that can occur during fine-tuning, and does not indicate a problem with the training process. Validation loss increased slightly after epoch 1, which is common when fine-tuning pre-trained models on a fixed dataset, the model continues to improve its predictions on the training data while the validation loss stabilises.

The gap between training accuracy (98.17%) and validation accuracy (94.62%) at epoch 3 is expected and does not indicate severe overfitting, given that the validation performance remained strong throughout training. The best model checkpoint was saved at epoch 3, which produced the highest validation accuracy.

As illustrated in Figure 28, the training loss curve decreases steadily across all three epochs, while the validation loss curve stabilises after epoch 1. The accuracy curves show a consistent improvement in training accuracy alongside a stable validation accuracy, confirming that the model generalised well to unseen data.

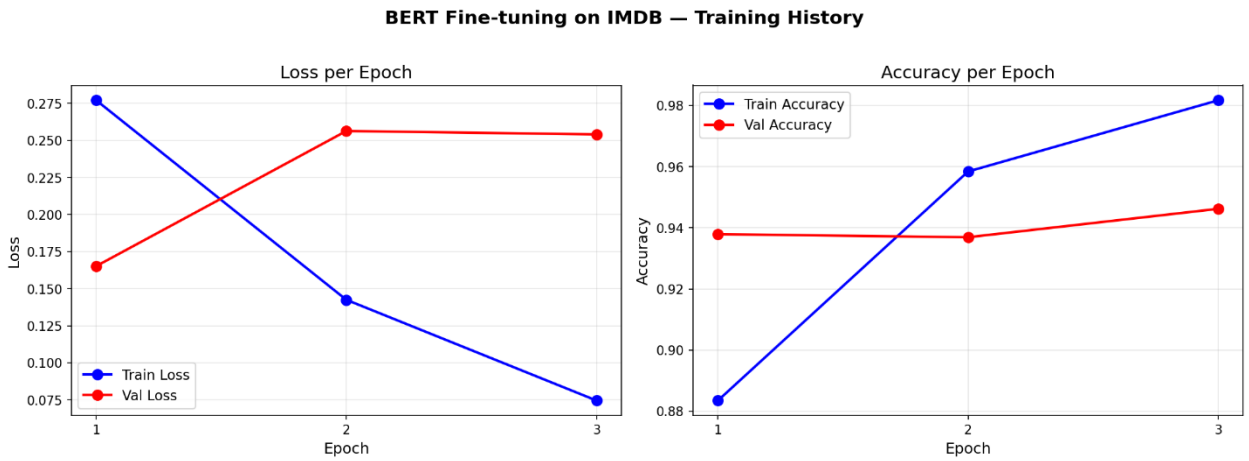


Figure 28: BERT Fine-tuning on IMDB Training History

6.2 MAX_LENGTH Comparison

To determine the optimal maximum sequence length for tokenization, three configurations were evaluated on the validation set: 128, 256, and 512 tokens. The results are presented in Table 7.

Table 7: Validation results for each MAX_LENGTH configuration

MAX_LENGTH	Val Accuracy	Val F1 Score	Epoch 1 time	Epoch 2 time	Epoch 3 time	Total train time
128	89.90%	90.06%	15.2 min	15.4 min	15.4 min	50.9 min
256	92.77%	92.80%	30 min	29.9 min	30 min	97.4 min
512	94.26%	94.33%	53.8 min	53.8 min	53.8 min	174.1 min

As illustrated in Figure 29, both validation accuracy and F1 score increase consistently as the maximum sequence length grows. The improvement from 128 to 256 tokens is larger than the improvement from 256 to 512 tokens, which suggests that the gains become smaller as the sequence length approaches the maximum supported by BERT.

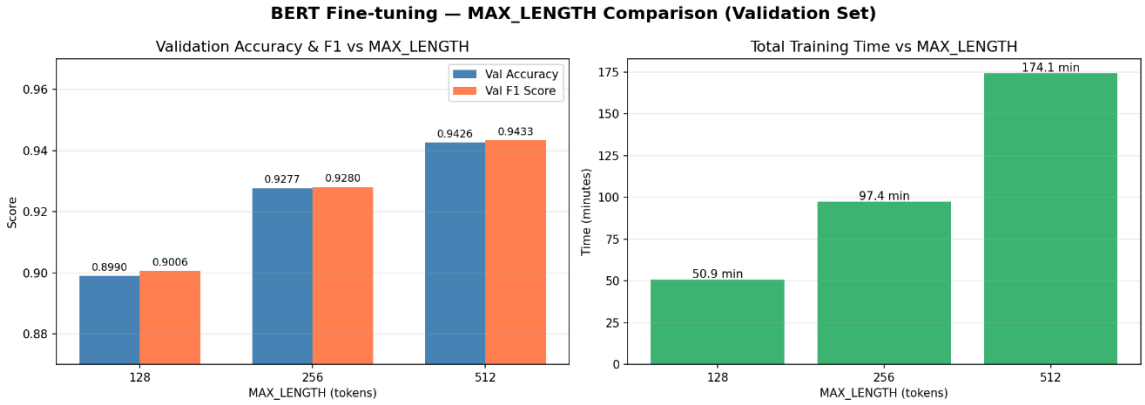


Figure 29: BERT Fine-tuning MAX_LENGTH Comparison (Validation Set)

One possible explanation for the superior performance of longer sequences is directly related to the nature of the IMDB dataset. Movie reviews are often long and detailed, with sentiment expressed across multiple paragraphs. When the maximum sequence length is set to 128 or 256 tokens, a significant portion of each review is truncated, and the model loses access to sentiment-bearing content that appears later in the text. With MAX_LENGTH=512, the model can process a much larger portion of each review, giving it more context to make an accurate prediction.

The trade-off is training time. Moving from 128 to 512 tokens increases training time from 50.9 minutes to 174.1 minutes, approximately four times longer.

Based on these results, MAX_LENGTH=512 was selected as the final configuration, as it produced the highest validation accuracy and F1 score. This choice was made exclusively on the basis of validation set performance, and the test set was not consulted at any point during this experiment.

6.3 Final Evaluation Results

After the MAX_LENGTH comparison experiment confirmed that 512 tokens produced the best validation performance, the final model trained with this configuration was evaluated on the held-out test set. The test set of 7,437 samples was used once, and the results are presented in Table 8.

Table 8: Final evaluation results on the test set

Metric	Value
Accuracy	94.50%
Precision	93.90%
Recall	95.23%
F1 Score	94.56%

The model achieved an accuracy score of 94.50% and F1 score of 94.56% on the test set. The close agreement between accuracy and F1 score confirms that the model performs consistently across both classes, with no significant bias toward positive or negative predictions. The small difference between precision (93.90%) and recall (95.23%) indicates that the model is slightly better at avoiding false positives than false negatives, but the gap is minimal and does not affect the overall results.

These results are above the values reported in recent literature on the same dataset. Puspita and Rahayu [7] reported a test accuracy of 91.78% with a fine-tuned BERT-base model, Islam et al. [3] reported BERT as the best performing architecture in their comparison with an accuracy below 92%, and Popescu et al. [8] reported a BERT accuracy of 92% and F1 score of 92% under identical experimental conditions across five transformer models. The configuration considered in this paper surpasses the above-mentioned results.

This improvement may be attributed to several factors. First, the use of the full 512-token sequence length allows the model to process more of each review compared to studies that use shorter truncation thresholds. Second, the stratified three-way split ensures that model selection decisions were made exclusively on the validation set, with the test set consulted only once, reducing the risk of implicit overfitting to the test data. Third, the combination of AdamW with weight decay, linear warmup scheduling, and gradient clipping provides a stable and well-regularised training setup that helps the model generalise effectively. Nevertheless, direct comparison with prior studies should be interpreted cautiously, as experimental protocols, preprocessing steps and implementation details may differ.

The following two examples from Figure 30 test set illustrate cases where the model correctly identified the sentiment. The last two examples, shown in Figure 31 and Figure 32, demonstrate how accurately the deployed application predicts sentiment when given new input text through the graphical interface:

Review Positive (Label: 1):

"I would purchase this and "Thirty Seconds Over Tokyo" today if available. I also saw this movie after seeing large billboards of Jack Webb in his Dress Blues on the highway at the age of 12. Always admired Jack Webb as a John Wayne type and American all the way. Almost became a Marine just because of this movie but served instead in the Air Force, Air Force Reserve and Army Reserve for 32 years. Have not seen this movie on TV at all and would love to own it on DVD. At least if it was on TV I would copy it to VHS and use it until available on DVD. I also have a large collection of WWII and Korean War movies and always look for new releases on DVD."

Review Negative (Label: 0):

"Lesbian vampire film about a couple on holiday who are staying on the grounds of what they think is an empty manor house but is really being used as a pair of lesbian vampires. Great looking film with two very sexy women as the vampires, there is nothing beyond the eye candy that they provide to recommend this cult film. Yes its a sexy vampire story. No it is not remotely interesting beyond the women. To be honest there is a reason that I've been seeing stills of this film in horror books and magazines — it looks great, but other than that... For those who want to see sexy vampires only."

Figure 30. Test set reviews

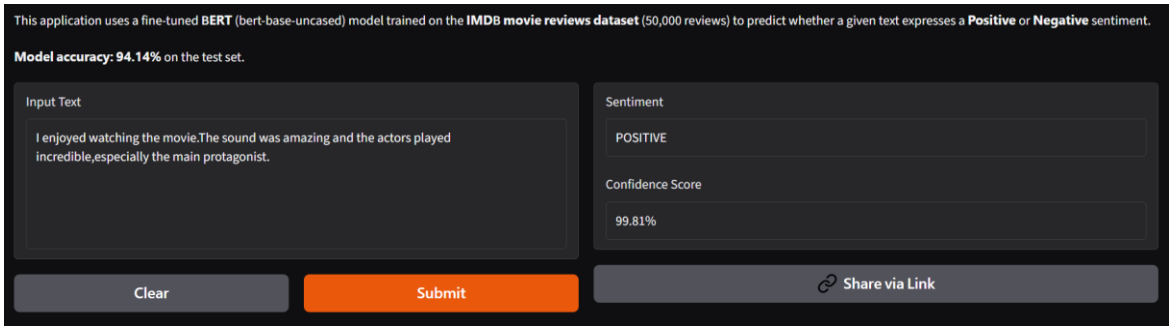


Figure 31. BERT Sentiment Analyzer: Positive sentiment prediction with 99.81% confidence score.

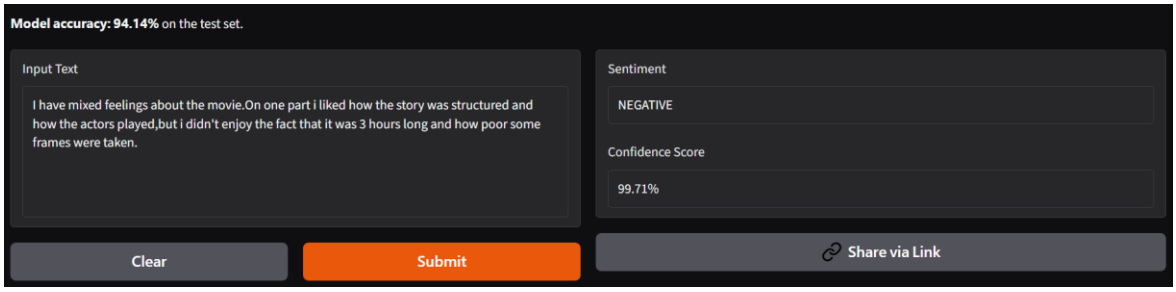


Figure 32. BERT Sentiment Analyzer: Negative sentiment prediction with 99.71% confidence score.

6.4 Prediction Analysis

6.4.1 Confusion Matrix

The confusion matrix obtained on the test set is shown in Figure 33. Out of 7,437 test samples, the model correctly classified 3,474 negative reviews (True Negatives) and 3,554 positive reviews (True Positives). It incorrectly classified 231 negative reviews as positive (False Positives) and 178 positive reviews as negative (False Negatives).

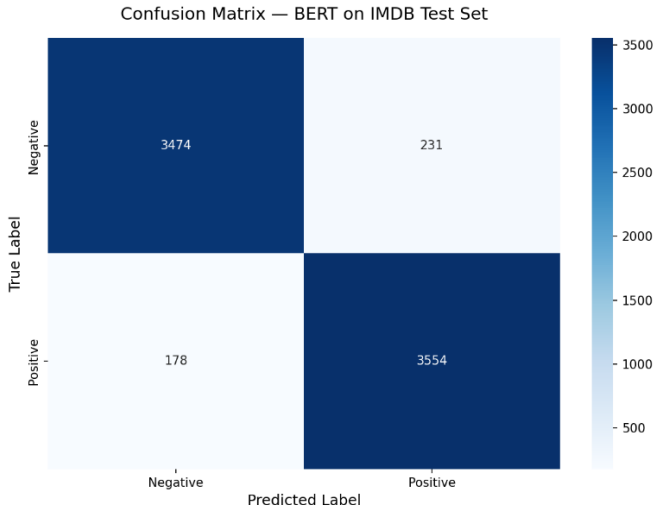


Figure 33: Confusion Matrix BERT on IMDB Test Set

The distribution of errors is relatively balanced between the two classes. The number of false positives (231) and false negatives (178) is similar, which confirms that the model does not have a strong directional bias toward either class. The slightly higher number of false positives (231) compared to false negatives (178) suggests that the model is marginally more likely to predict positive sentiment, but the difference is small and does not affect the overall reliability of the results.

The high values on the main diagonal of the confusion matrix, 3,474 and 3,554, compared to the off-diagonal values, 231 and 178, confirm that the model classifies the large majority of reviews correctly for both classes, with consistent performance across positive and negative sentiment.

6.4.2 Error Analysis

To better understand where the model makes mistakes, an error analysis was performed on the 409 misclassified reviews, representing an error rate of 5.50% out of 7,437 test samples.

The first observation concerns review length. Figures 34 and 35 show the review length distributions for correctly classified and misclassified reviews, respectively. Misclassified reviews are slightly longer on average, with a mean length of 243 words compared with 228 words for correctly classified reviews. This association suggests that review length may contribute to classification difficulty. One possible explanation is truncation, since longer reviews are more likely to exceed the 512 token input limit, potentially causing relevant sentiment information to be lost.

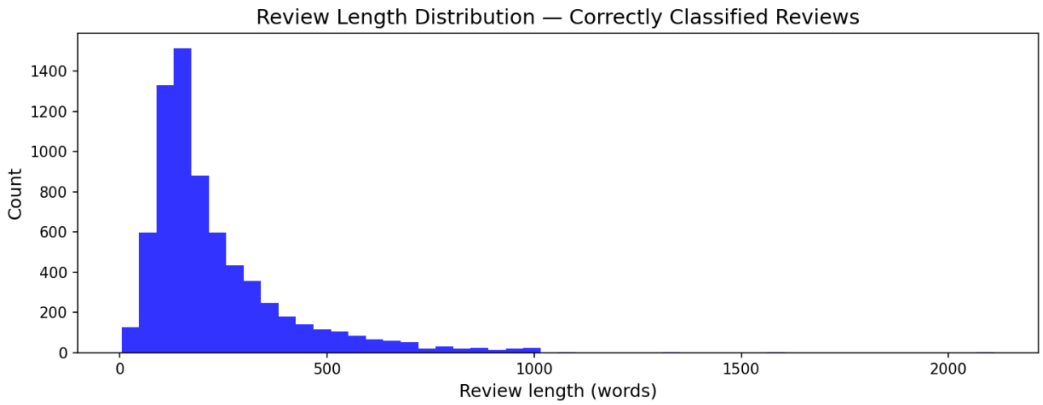


Figure 34: Review Length, correctly classified reviews.

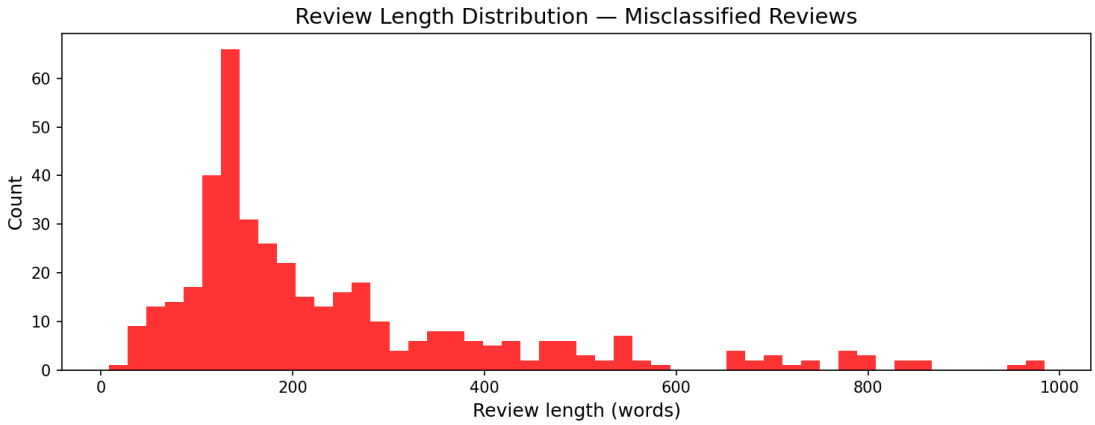


Figure 35: Review Length, misclassified reviews.

The second observation concerns reviews containing potential irony or sarcasm-related keywords. The heuristic keyword set included: supposedly, apparently, brilliant, genius, masterpiece, wow, amazing, sure, right, of course, obviously, great job, well done. Among the 3,477 reviews matching this heuristic, 203 were misclassified, corresponding to a misclassification rate of 5.8%, slightly higher than the overall rate of 5.50%. While this does not demonstrate that irony is the cause of these errors, it suggests that reviews containing such expressions may be somewhat more challenging for the model, as the surface level words may suggest a different sentiment than the intended meaning.

To complement the quantitative analysis, three misclassified reviews illustrating different error patterns are presented in Figure 36. The following examples were manually selected from the misclassified reviews to illustrate recurring qualitative error patterns.

Example #7 — True: Negative, Predicted: Positive

“Scooby Doo is undoubtedly one of the most simple, successful and beloved cartoon characters in the world. So, what happens when you’ve been everywhere and done everything with the formula? You switch it up right? Wrong. You stop production and let it rest for a decade or so and then run it again, keeping the core of its success intact. That is to say, stick with the formula for the most part but adapt it for a new generation.”

Example #41 — True: Positive, Predicted: Negative

“Just watched it on Sky TV, missed the first half an hour. I did wonder if it was a true story so watched it to the end. There was no brief at the end to say what happened to everyone. It did remind me of Speed, but at the end of the day I don’t suppose it was released at the cinema as we did see

the following error — the goof that we saw was that they remove the bonnet and then later on it reappears."

Example #103 — True: Positive, Predicted: Negative

"I loved the movie Northfork. I knew nothing about the movie before watching it. Therefore, I had no outside influence or information to guide me in what I was seeing unfold on the screen. In retrospect I would advise anyone interested in the movie to watch it if for no other reason than the quality actors who appear in it. Do not read anything about the plot, story line, or evaluation of the movie."

Figure 36. Examples of misclassified reviews.

A third pattern observed among the analysed errors involves reviews that contain both positive and negative statements within the same text, called mixed sentiment. In these cases, the reviewer expresses praise for certain aspects of the work while criticising others, creating a mixed signal that the model must resolve into a single binary label. This type of ambiguity is particularly challenging because both sentiment polarities are present and the overall label depends on which aspect the reviewer considers more important.

Example #7 from the test set illustrates this pattern. The review begins with strong positive language, describing Scooby Doo as one of the most successful and beloved cartoon characters, before shifting to criticism of the creative decisions made in the adaptation. The model predicted Positive with 99.68% confidence, while the true label is Negative. The reviewer intended the review as a negative assessment of the adaptation, but the abundance of positive vocabulary in the opening sentences overwhelmed the model's ability to detect the overall negative conclusion.

To further investigate this pattern, a mixed sentiment input was tested through the deployed Gradio application: "The plot was weak and predictable, but the actors delivered outstanding performances." The model classified this input as Positive with 99.54% confidence, as shown in Figure 37. This result confirms that BERT tends to resolve mixed sentiment in favour of whichever polarity is expressed with stronger vocabulary, regardless of the overall intent.

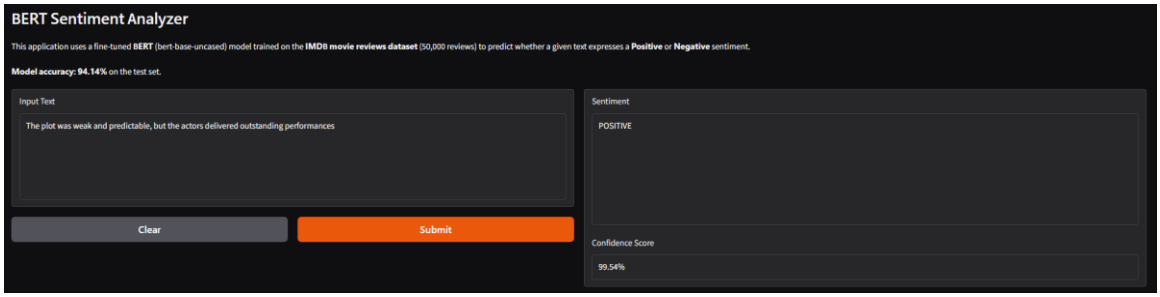


Figure 37. BERT Sentiment Analyzer: Mixed sentiment input classified as Positive with 99.54% confidence.

A fourth category of errors involves reviews that express a mild or ambivalent opinion without strong sentiment-bearing vocabulary, called weak sentiment. These reviews are characterised by hedging language, qualified statements, and an overall tone that is neither clearly positive nor clearly negative. The absence of strong opinion words makes it difficult for the model to identify a clear sentiment signal.

Example #41 from the test set demonstrates this pattern. The review describes watching a movie on television in a casual, observational tone, mentioning that it reminded the reviewer of another film and noting a production error. The model predicted Negative with 98.67% confidence, while the true label is Positive. The reviewer intended the review as a mildly positive assessment, but the low-enthusiasm language and the mention of a production error led the model to classify it as negative.

To test this pattern further, a weak sentiment input was submitted to the Gradio application: "It was ok, nothing special but not terrible either." The model classified this input as Negative with 99.63% confidence, as shown in Figure 38. This result illustrates that BERT interprets the absence of positive vocabulary as evidence of negative sentiment, even when the reviewer's intent is closer to neutral.

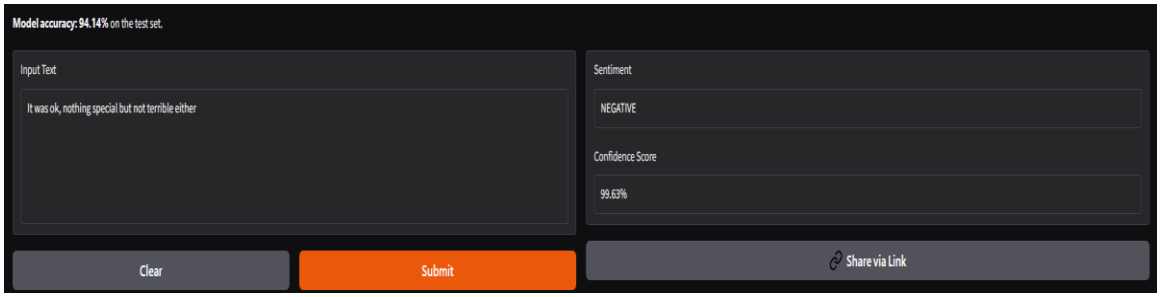


Figure 38. BERT Sentiment Analyzer: Weak sentiment input classified as Negative with 99.63% confidence.

These misclassification patterns can be explained by how BERT processes language. Although BERT captures bidirectional context through its attention mechanism, it learns associations between words and sentiment labels from the training data. When a review uses indirect language, irony, or sarcasm, the individual words may carry a different sentiment than the overall meaning intended by the author. BERT may struggle in these cases because it has not seen enough examples of this type of language during training to learn the correct association.

A similar issue occurs with long reviews. When a review exceeds 512 tokens, the text is truncated and the model only sees the first 512 tokens. If the most sentiment-bearing part of the review appears toward the end of the text, this information is lost during tokenization, making it harder for the model to assign the correct label.

These are known limitations of transformer-based models in general, and not specific to this implementation. Addressing them would require either a longer context window, additional training data containing ironic and sarcastic examples, or a post-processing step that handles these edge cases explicitly.

6.4.3 Model Confidence Analysis

To gain deeper insight into the nature of misclassifications, a confidence analysis was performed on the misclassified examples discussed in the previous section. For each example, the softmax probabilities were computed from the raw logits produced by the model, yielding a confidence score that represents the probability assigned by the model to its predicted class. This analysis reveals whether the model was uncertain about its incorrect predictions, which would suggest that it recognised the ambiguity in the input, or whether it was highly confident in the wrong answer, which would indicate a more fundamental misunderstanding of the text.

Table 9 presents the confidence scores for the three misclassified examples discussed in Section 6.4.2, alongside Example #50 which exhibited notably different confidence behaviour. For comparison, as shown in Figure 39, Example #50 was selected as a low-confidence misclassification:

Example #50 — True: Negative, Predicted: Positive

"Arthur Hunnicutt plays a very stereotypical role as a mountain man (probably the Ozarks) who goes hunting with his favorite coon dog. However, the dog appears to be drowning when Hunnicutt jumps in after him. It becomes obvious pretty soon that despite Hunnicutt and his dog roaming about after leaving the water that they both died in the water--as no one responds when he talks to them and sees and"

Figure 39. Example of a low-confidence misclassification

Table 9: Model confidence scores for selected misclassified examples

Example	True Label	Predicted Label	Confidence	Error Category
#7	Negative	Positive	99.68%	Mixed Sentiment
#41	Positive	Negative	98.67%	Weak Sentiment
#103	Positive	Negative	99.41%	Irony/Sarcasm
#50	Negative	Positive	53.45%	Low Confidence

The results reveal that for three out of four analysed examples, the model was not merely uncertain but was highly confident in its incorrect prediction, with confidence scores exceeding 98%.

- Example #7 was classified as Positive with 99.68% confidence despite being a negative review
- Example #41 was classified as Negative with 98.67% confidence despite being positive
- Example #103 was classified as Negative with 99.41% confidence despite being positive.

These high-confidence errors indicate that the model did not recognise the ambiguity in these inputs. Instead, it processed the surface-level vocabulary and arrived at a definitive, incorrect, conclusion.

The exception is Example #50, where the model predicted Positive with only 53.45% confidence, slightly above the 50% decision boundary. This near-boundary confidence indicates that the model assigned similar probabilities to the two classes and was therefore substantially less certain than in the other analysed cases. In contrast to the high-confidence errors, this example illustrates a prediction for which the model showed considerable uncertainty but was still required to assign one of the two labels within the binary classification framework.

The presence of high-confidence errors among the selected examples has practical implications. A confidence threshold, for example rejecting predictions below 80% confidence, would not have caught Examples #7, #41, or #103, since the model was more than 98% confident in all three cases. This suggests that confidence-based filtering alone is insufficient to detect misclassifications in cases involving mixed sentiment, weak sentiment, or indirect language. Potential directions for addressing such cases include evaluating models with different architectures or incorporating additional training data that better represents mixed, weak and indirect sentiment expressions.

7. Conclusion

This work presented the development and evaluation of a sentiment analysis system based on the fine-tuning of the BERT transformer model on the IMDB Large Movie Review Dataset. The primary objective was to build a well-documented and reproducible pipeline that achieves strong classification performance on this benchmark, while following best practices in terms of data splitting, model selection, and evaluation.

The preprocessing pipeline addressed several data quality issues identified during exploratory data analysis, including the presence of HTML tags, whitespace artifacts, and duplicate entries. After deduplication, the dataset was reduced from 50,000 to 49,578 reviews and partitioned into training, validation, and test sets using a stratified split strategy, ensuring consistent class distribution across all three subsets. The test set was reserved exclusively for final evaluation and was not used at any point during model development.

The model was fine-tuned using the bert-base-uncased checkpoint with a carefully selected set of hyperparameters, including an AdamW optimiser with a learning rate of 2×10^{-5}, weight decay of 0.01, a linear warmup schedule over the first 10% of training steps, and gradient clipping with a maximum norm of 1.0. A dedicated MAX_LENGTH comparison experiment was conducted on the validation set to determine the optimal sequence length, showing that MAX_LENGTH=512 produced the best results, with a validation accuracy of 94.26%, compared to 92.77% for MAX_LENGTH=256 and 89.90% for MAX_LENGTH=128.

The final model was evaluated on the test set and achieved an accuracy and F1 score of 94.56%, a precision of 93.90%, and a recall of 95.23%. These results are higher than those reported in the recent studies selected for comparison in this work [3][7][8]. The confusion matrix confirmed that the model performs consistently across both classes, with 3,474 true negatives and 3,554 true positives out of 7,437 test samples, and a relatively balanced distribution of errors between false positives (231) and false negatives (178).

The error analysis revealed several patterns among the misclassified examples. Misclassified reviews were, on average, longer than correctly classified ones, while the qualitative inspection of individual errors highlighted challenges involving mixed sentiment, indirect formulations, and conflicting sentiment cues. The confidence analysis further showed that most of the selected errors were high-confidence predictions, suggesting that the model can sometimes form strong but incorrect interpretations of complex reviews.

The fine-tuned model was deployed as an interactive web application using the Gradio library and hosted on Hugging Face Spaces, where it is accessible 24/7 through a public URL. The application allows users to input a review and receive a sentiment prediction along with a confidence score.

In the development of this degree, generative AI tools, including Claude, were used as instruments to support certain aspects of the writing process, including improving text readability, reformulating existing content, and generating initial drafts of specific sections.

Future work could explore several directions based on the patterns observed in this study. The influence of review length and truncation could be investigated more systematically by comparing performance on truncated and non-truncated reviews or by evaluating models with longer context windows, such as Longformer or BigBird. Further analysis could also examine systematically defined categories of difficult reviews, including mixed sentiment, indirect formulations and conflicting sentiment cues. Additionally, ensemble methods combining multiple transformer models could be explored as a potential direction for further performance improvement.

8. References

[1] M. Wankhade, A. C. S. Rao, and C. Kulkarni, "A survey on sentiment analysis methods, applications, and challenges," *Artificial Intelligence Review*, vol. 55, no. 7, pp. 5731–5780, Oct. 2022. https://link.springer.com/article/10.1007/s10462-022-10144-1

[2] "Traditional and Deep Learning Approaches for Sentiment Analysis: A Survey," ResearchGate, 2021. https://www.researchgate.net/publication/354800377

[3] M. T. Islam, F. Parvin, S. A. Sazan, and T. B. Amir, "Comparative analysis of sentiment classification on IMDb 50k movie reviews: A study using CNN, LSTM, CNN-LSTM, and BERT models," in *Proc. IEEE PEEIACON*, 2024, pp. 512–517.

[4] M. M. Danyal et al., "Proposing sentiment analysis model based on BERT and XLNet for movie reviews," *Multimedia Tools and Applications*, vol. 83, no. 24, pp. 64315–64339, 2024. https://link.springer.com/article/10.1007/s11042-023-17879-3

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, Jun. 2019, pp. 4171–4186. https://arxiv.org/abs/1810.04805

[6] A. Areshey and H. Mathkour, "Exploring transformer models for sentiment classification," *Expert Systems*, 2024. https://onlinelibrary.wiley.com/doi/10.1111/exsy.13701

[7] R. Puspita and C. Rahayu, "Sentiment analysis on IMDB movie reviews using BERT," *Indonesian Journal of Artificial Intelligence and Data Mining*, Dec. 2023. https://www.researchgate.net/publication/377024012

[8] A. Popescu et al., "TWSSenti: A novel hybrid framework for topic-wise sentiment analysis," *arXiv*:2504.09896, 2025. https://arxiv.org/abs/2504.09896

[9] "A Resource-Efficient Approach to Fine-Tuning a BERT-Base Model for Sentiment Analysis," *Computers*, vol. 15, no. 3, p. 159, 2026. https://www.mdpi.com/2073-431X/15/3/159

[10] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. NIPS*, 2013, pp. 3111–3119. https://arxiv.org/abs/1310.4546

[11] J. Pennington, R. Socher, and C. Manning, "GloVe: Global vectors for word representation," in *Proc. EMNLP*, 2014, pp. 1532–1543. https://aclanthology.org/D14-1162

[12] A. Vaswani et al., "Attention is all you need," in *Proc. NIPS*, 2017, pp. 5998–6008. https://arxiv.org/abs/1706.03762

[13] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," 2019. https://arxiv.org/abs/1907.11692

[14] V. Sanh et al., "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," 2019. https://arxiv.org/abs/1910.01108

[15] Z. Yang et al., "XLNet: Generalized autoregressive pretraining for language understanding," in *Proc. NeurIPS*, 2019. https://arxiv.org/abs/1906.08237

[16] A. L. Maas et al., "Learning word vectors for sentiment analysis," in *Proc. ACL*, 2011, pp. 142–150. https://aclanthology.org/P11-1015

[17] "Confusion Matrix," ResearchGate. [Online]. Available: https://www.researchgate.net/figure/Confusion-Matriks-The-confusion-matrix-is-a-technique-for-evaluating-the-performance-of-a_fig1_371624916

9. Annexes

```
#
================================================================================
# ANNEX 1: analiza_sentimente.ipynb – Exploratory Data Analysis
#
================================================================================

# Thesis Project: Sentiment Analysis using Transformer Models

# Phase 1: Data Loading, Cleaning, and Exploratory Data Analysis (EDA)
# 1. Data Loading and Initial Inspection
# In this section, we load the IMDB Dataset and perform an initial screening of
its structure. The dataset consists of 50,000 movie reviews, labeled as either
positive or negative.
# 2. Class Distribution Analysis
# It is crucial to check for class balance to ensure the model does not develop a
bias during the training process.
# 3. Data Splitting
# We split the dataset into three distinct subsets: Training (80%), Validation
(10%), and Testing (10%). We apply stratification based on the sentiment label to
maintain an identical class distribution across all subsets.

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split

# 1. Data Loading
df = pd.read_csv('IMDB Dataset.csv')

# 2. Check total number of reviews
print(f"Total number of reviews: {len(df)}")

# 3. Check class distribution
print("\nClass distribution:")
print(df['sentiment'].value_counts())

# Graphical visualization of the distribution
sns.countplot(x='sentiment', data=df)
plt.title('Sentiment Distribution (Positive vs Negative)')
plt.show()

# 4. Splitting into Train (80%), Val (10%), and Test (10%)
# Stratify ensures class balance is maintained across all sets
train_df, temp_df = train_test_split(df, test_size=0.20, random_state=42,
stratify=df['sentiment'])
val_df, test_df = train_test_split(temp_df, test_size=0.50, random_state=42,
stratify=temp_df['sentiment'])
```

```
print(f"\nFinal dimensions:")
print(f"Training set (Train): {len(train_df)}")
print(f"Validation set (Val): {len(val_df)}")
print(f"Testing set (Test): {len(test_df)}")

# 4. Text Length Analysis
# We analyze the length of the reviews to understand the input constraints for
our model. Since models like BERT have a maximum token limit (usually 512),
understanding the distribution of review lengths is essential for deciding on a
strategy.

# 1. Display examples (review + label)
print("--- Review Examples ---")
pd.set_option('display.max_colwidth', 200) # Allow more text visibility
print(train_df.sample(5)[['review', 'sentiment']])

# 2. Issue Analysis (Missing values, Empty texts, Duplicates)
print("\n--- Data Quality Check ---")
print(f"Missing values:\n{train_df.isnull().sum()}")

# Check for texts that contain only whitespace
empty_texts = train_df[train_df['review'].str.strip() == ""].shape[0]
print(f"Empty texts (whitespace only): {empty_texts}")

# Check for duplicates (the same review written multiple times)
duplicates = train_df.duplicated(subset=['review']).sum()
print(f"Duplicate reviews: {duplicates}")

# 3. Text length analysis
# Calculate the number of characters for each review
train_df['review_len'] = train_df['review'].apply(len)

print("\n--- Text Length Statistics (Characters) ---")
print(train_df['review_len'].describe())

# Visualize length distribution
plt.figure(figsize=(10, 5))
sns.histplot(train_df['review_len'], bins=50, kde=True)
plt.title('Review Length Distribution')
plt.xlabel('Number of Characters')
plt.ylabel('Frequency')
plt.show()

# 5. Data Cleaning and Deduplication
# Initial EDA revealed the presence of HTML tags (e.g., <br />) and duplicate
entries. In this stage, we perform the following preprocessing steps:
# * HTML Removal:Tags that do not provide semantic value.
# * Whitespace Normalization: Removing redundant spaces.
# * Deduplication: Removing identical reviews from the training set to prevent
the model from memorizing specific examples
```

```
import re

def clean_text(text):
    # 1. Remove HTML tags (e.g., <br />)
    text = re.sub(r'<.*?>', ' ', text)
    # 2. Remove resulting multiple whitespaces
    text = re.sub(r'\s+', ' ', text).strip()
    return text

# Apply cleaning to all segments
train_df['review'] = train_df['review'].apply(clean_text)
val_df['review'] = val_df['review'].apply(clean_text)
test_df['review'] = test_df['review'].apply(clean_text)

# Remove duplicates from the Training set
train_df = train_df.drop_duplicates(subset=['review'])

print("Cleaning completed!")
print(f"Total training samples after deduplication: {len(train_df)}")

# 6. Final Observations (EDA Summary):
# 1. Class Balance: The dataset is perfectly balanced (50/50), eliminating the
need for oversampling or undersampling techniques.
# 2. Data Quality: The text has been cleaned of HTML noise and duplicates. No
missing (null) values were detected.
# 3. Technical Constraints: Several reviews exceed 10,000 characters. These will
be subject to truncation at the 512-token threshold during the tokenization phase
to fit the Transformer's context window.



#
================================================================================
# ANNEX 2: preprocessing.ipynb — Data Preprocessing Pipeline
#
================================================================================

# ============================================================================
# preprocess.py
# Data preprocessing and tokenization pipeline for IMDB sentiment analysis
# ============================================================================

import os
import re
import pandas as pd
from datasets import load_dataset, Dataset, DatasetDict
from transformers import BertTokenizer
from sklearn.model_selection import train_test_split

# ============================================================================
# CONFIGURATION
```

```python
# ============================================================================

TOKENIZER_NAME  = "bert-base-uncased"
MAX_LENGTH      = 512       # determined from EDA: covers ~95% of reviews
TEST_SIZE       = 0.15      # 15% test
VAL_SIZE        = 0.15      # 15% validation (from the remaining 85%)
RANDOM_STATE    = 42
SAVE_PATH       = "data/tokenized_imdb"

# ============================================================================
# STEP 1 — LOAD RAW DATA
# ============================================================================

def load_raw_data():
    """Load the IMDB dataset from HuggingFace hub."""
    print("[1/5] Loading IMDB dataset...")
    raw = load_dataset("imdb")

    # Combine train + test splits into a single DataFrame for custom splitting
    train_df = pd.DataFrame(raw["train"])
    test_df  = pd.DataFrame(raw["test"])
    df = pd.concat([train_df, test_df], ignore_index=True)

    print(f"   Total samples loaded: {len(df)}")
    return df

# ============================================================================
# STEP 2 — CLEAN + LABEL ENCODE
# ============================================================================

def clean_text(text):
    """Remove HTML tags and normalize whitespace."""
    text = re.sub(r"<.*?>", " ", text)        # remove HTML tags
    text = re.sub(r"\s+", " ", text).strip()  # collapse whitespace
    return text

def clean_and_encode(df):
    """Clean text and encode labels: positive -> 1, negative -> 0."""
    print("[2/5] Cleaning text and encoding labels...")

    df["text"]  = df["text"].apply(clean_text)

    # Drop duplicates identified during EDA
    before = len(df)
    df = df.drop_duplicates(subset=["text"]).reset_index(drop=True)
    print(f"   Removed {before - len(df)} duplicate reviews.")

    # Labels are already 0/1 in the IMDB dataset (0=neg, 1=pos) — verify
    print(f"   Label distribution:\n{df['label'].value_counts()}")
    return df
```

```
# ============================================================================
# STEP 3 — STRATIFIED TRAIN / VALIDATION / TEST SPLIT
# ============================================================================

def split_data(df):
    """
    Stratified split:
      - 70% train  | 15% validation  | 15% test
    Stratification ensures equal class ratios in every split.
    """
    print("[3/5] Splitting dataset (70 / 15 / 15) with stratification...")

    # First split: separate test set
    train_val_df, test_df = train_test_split(
        df,
        test_size=TEST_SIZE,
        random_state=RANDOM_STATE,
        stratify=df["label"]
    )

    # Second split: separate validation from remaining data
    relative_val_size = VAL_SIZE / (1.0 - TEST_SIZE)   # ~0.176
    train_df, val_df = train_test_split(
        train_val_df,
        test_size=relative_val_size,
        random_state=RANDOM_STATE,
        stratify=train_val_df["label"]
    )

    print(f"   Train : {len(train_df):>6} samples")
    print(f"   Val   : {len(val_df):>6} samples")
    print(f"   Test  : {len(test_df):>6} samples")
    return train_df, val_df, test_df

# ============================================================================
# STEP 4 — TOKENIZATION
# ============================================================================

def tokenize_dataset(train_df, val_df, test_df):
    """
    Tokenize all splits using BertTokenizer.
    - truncation=True  : cut sequences longer than MAX_LENGTH
    - padding='max_length': pad shorter sequences to MAX_LENGTH
    """
    print(f"[4/5] Loading tokenizer '{TOKENIZER_NAME}'...")
    tokenizer = BertTokenizer.from_pretrained(TOKENIZER_NAME)

    def tokenize_fn(batch):
        return tokenizer(
            batch["text"],
            truncation=True,
```

```
            padding="max_length",
            max_length=MAX_LENGTH,
        )

    # Convert DataFrames -> HuggingFace Datasets
    dataset_dict = DatasetDict({
        "train" : Dataset.from_pandas(train_df[["text", "label"]]),
        "val"   : Dataset.from_pandas(val_df[["text", "label"]]),
        "test"  : Dataset.from_pandas(test_df[["text", "label"]]),
    })

    print("   Tokenizing all splits (this may take 1-2 minutes)...")
    tokenized = dataset_dict.map(
        tokenize_fn,
        batched=True,
        batch_size=500,
        desc="Tokenizing",
    )

    # Set PyTorch format: keep only the columns the model needs
    tokenized = tokenized.remove_columns(["text"])
    tokenized = tokenized.rename_column("label", "labels")
    tokenized.set_format("torch")

    return tokenized

# ============================================================================
# STEP 5 — SAVE / LOAD
# ============================================================================

def save_tokenized(tokenized):
    """Save the tokenized DatasetDict to disk."""
    print(f"[5/5] Saving tokenized dataset to '{SAVE_PATH}'...")
    os.makedirs(SAVE_PATH, exist_ok=True)
    tokenized.save_to_disk(SAVE_PATH)
    print("   Done! Dataset saved successfully.")

def load_tokenized():
    """Load the pre-tokenized DatasetDict from disk."""
    from datasets import load_from_disk
    print(f"Loading tokenized dataset from '{SAVE_PATH}'...")
    return load_from_disk(SAVE_PATH)

# ============================================================================
# MAIN
# ============================================================================

if __name__ == "__main__":

    if os.path.exists(SAVE_PATH):
        print(f"Tokenized dataset already exists at '{SAVE_PATH}'.")
```

```
        print("   Delete the folder and re-run to recompute.")
    else:
        df          = load_raw_data()
        df          = clean_and_encode(df)
        train_df, val_df, test_df = split_data(df)
        tokenized   = tokenize_dataset(train_df, val_df, test_df)
        save_tokenized(tokenized)

    # Quick sanity check — load and print one sample
    dataset = load_tokenized()
    sample  = dataset["train"][0]
    print("\n--- Sanity Check (first training sample) ---")
    print(f"  input_ids shape      : {sample['input_ids'].shape}")
    print(f"  attention_mask shape : {sample['attention_mask'].shape}")
    print(f"  token_type_ids shape : {sample['token_type_ids'].shape}")
    print(f"  label                : {sample['labels'].item()}")



#
# ANNEX 3: model.ipynb — BERT Model Loading and Sanity Check
#

# BERT Model Loading
# Loading pre-trained BERT for binary sentiment classification (positive /
negative)

# Imports

# Core libraries for model loading and dataset handling
import torch
from transformers import BertForSequenceClassification, BertTokenizer
from datasets import load_from_disk

# Configuration

# Model configuration
MODEL_NAME  = "bert-base-uncased" # pre-trained BERT variant (uncased = lowercase
NUM_LABELS  = 2          # binary: positive / negative
SAVE_PATH   = "data/tokenized_imdb"

# Detect device: use GPU if available, otherwise CPU
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {DEVICE}")

# Step 1 — Load Tokenized Dataset
# Loading the pre-tokenized dataset saved during preprocessing (no re-
tokenization needed).
```

```
# Load the pre-tokenized dataset saved during preprocessing
# Avoids re-running the tokenization pipeline on every execution
dataset = load_from_disk(SAVE_PATH)
print(dataset)

# Step 2 — Load BERT Model
# Loading `BertForSequenceClassification` with a classification head on top of
BERT.
# - **UNEXPECTED keys**: MLM head from pre-training — safely ignored
# - **MISSING keys**: new classification head — randomly initialized, will be
trained

# Load pre-trained BERT for binary sequence classification
# num_labels=2 adds a classification head on top of BERT
model = BertForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=NUM_LABELS,
)

# Move model to GPU if available
model = model.to(DEVICE)

print(f"\nModel loaded successfully!")
print(f"Model parameters: {model.num_parameters():,}")

# Step 3 — Sanity Check
# Verifying the model produces valid outputs on a real batch from the dataset.
# - Expected loss: ~0.693 (= ln(2)), which is normal for an untrained binary
classifier
# - Logits should be small and roughly equal before fine-tuning

# Verify model works with one real batch from the dataset
sample = dataset["train"][:2]

input_ids      = sample["input_ids"].to(DEVICE)
attention_mask = sample["attention_mask"].to(DEVICE)
labels         = sample["labels"].to(DEVICE)

# torch.no_grad() disables gradient computation — not needed for testing
with torch.no_grad():
    outputs = model(
        input_ids=input_ids,
        attention_mask=attention_mask,
        labels=labels
    )

print(f"Loss   : {outputs.loss.item():.4f}")
print(f"Logits: {outputs.logits}")
print(f"\nSanity check passed!")
```

```
#
# ANNEX 4: train.ipynb (Local) — DataLoaders Configuration
#

# BERT Fine-tuning — Training Pipeline
# Training and validation of BERT for sentiment analysis on the IMDB dataset
# **Model:** bert-base-uncased | **Epochs:** 3 | **Batch size:** 16

# Fix for Windows OpenMP conflict with PyTorch
import os
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
os.environ["OMP_NUM_THREADS"] = "1"

import torch
from torch.utils.data import DataLoader
from datasets import load_from_disk
from transformers import BertForSequenceClassification
print("Imports done!")

# Step 1 — Configuration

SAVE_PATH   = "data/tokenized_imdb"  # path to pre-tokenized dataset
MODEL_NAME  = "bert-base-uncased"    # pre-trained BERT variant
BATCH_SIZE  = 16                     # safe for both CPU and GPU
NUM_LABELS  = 2                      # binary: positive / negative

# Detect device: GPU significantly faster than CPU for training
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Device: {DEVICE}")

# Step 2 — DataLoaders
# Creating DataLoader objects for train, validation, and test splits.
# - **shuffle=True** for train: ensures different batch order each epoch
# - **shuffle=False** for val/test: order doesn't matter for evaluation

# Load tokenized dataset
dataset = load_from_disk(SAVE_PATH)

# Create DataLoaders for all 3 splits
train_loader = DataLoader(
    dataset["train"],
    batch_size=BATCH_SIZE,
    shuffle=True,        # shuffle train data each epoch
)

val_loader = DataLoader(
    dataset["val"],
    batch_size=BATCH_SIZE,
    shuffle=False,
```

```
)

test_loader = DataLoader(
    dataset["test"],
    batch_size=BATCH_SIZE,
    shuffle=False,
)

print(f"Train batches : {len(train_loader)}")
print(f"Val batches   : {len(val_loader)}")
print(f"Test batches  : {len(test_loader)}")

# Step 3 — Sanity Check DataLoader

# Verify one batch loads correctly
batch = next(iter(train_loader))
print(f"input_ids shape      : {batch['input_ids'].shape}")
print(f"attention_mask shape : {batch['attention_mask'].shape}")
print(f"labels shape         : {batch['labels'].shape}")
print(f"Labels in batch      : {batch['labels'][:8]}")
print("\nDataLoaders ready!")



#
================================================================================
# ANNEX 5: train.ipynb (Colab) — Training Pipeline
#
================================================================================

# BERT Fine-tuning on IMDB — Training & Validation
# Fine-tuning bert-base-uncased for binary sentiment classification
# **Dataset:** IMDB (49,578 reviews after cleaning) | **Split:** 70/15/15
# **Epochs:** 3 | **Batch size:** 16 | **Learning rate:** 2e-5 | **Device:** GPU
(T4)

# Step 1 — Imports & Setup

# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

pip install transformers datasets # Install the essential libraries for modeling
and data processing

import os
import random
import numpy as np
import torch

def set_seed(seed=42):
```

```
    """Fix all random seeds for full reproducibility."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    os.environ["PYTHONHASHSEED"] = str(seed)
    # Ensures deterministic behavior in cuDNN (GPU)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark     = False

set_seed(42)
print("Seed set to 42 — results are reproducible!")

# Step 2 — Data Loading and BERT Model Initialization
# We load the already tokenized dataset, set up the DataLoaders for training and
evaluation, and download the pre-trained BERT model, adapting its classification
head for 2 classes.

from datasets import load_from_disk
from transformers import BertForSequenceClassification
from torch.utils.data import DataLoader
import torch

# Path to tokenized dataset in Google Drive
SAVE_PATH  = "/content/drive/MyDrive/Licenta/tokenized_imdb"
MODEL_NAME = "bert-base-uncased" # Pre-trained BERT variant — uncased means all
text is lowercased
NUM_LABELS = 2 # Binary classification: 0 = Negative, 1 = Positive
DEVICE     = torch.device("cuda" if torch.cuda.is_available() else "cpu") # Use
GPU (T4 in Colab) if available, otherwise fall back to CPU

# Load dataset
dataset = load_from_disk(SAVE_PATH)

# DataLoaders
BATCH_SIZE = 16  # number of samples processed per forward pass

# shuffle=True for train: ensures different batch order each epoch
# shuffle=False for val/test: order doesn't affect evaluation results
train_loader = DataLoader(dataset["train"], batch_size=BATCH_SIZE, shuffle=True)
val_loader   = DataLoader(dataset["val"],   batch_size=BATCH_SIZE, shuffle=False)
test_loader  = DataLoader(dataset["test"],  batch_size=BATCH_SIZE, shuffle=False)

# Load model
model = BertForSequenceClassification.from_pretrained(MODEL_NAME,
num_labels=NUM_LABELS) # num_labels=2 replaces BERT's MLM head with a 2-class
classifier
model = model.to(DEVICE)

print(f"Device     : {DEVICE}")
print(f"Train batches : {len(train_loader)}")
```

```
print("Model ready!")

# Step 3 — Optimizer and Scheduler Setup (Learning Rate)
# We define how the model weights will be updated during training. We use AdamW
along with a scheduler that gradually increases the learning rate at the
beginning (warmup), then decreases it linearly.

from transformers import get_linear_schedule_with_warmup
from torch.optim import AdamW  # Using AdamW directly from PyTorch

# Hyperparameters
NUM_EPOCHS    = 3 # Total number of passes through the entire dataset
LEARNING_RATE = 2e-5 # Standard learning rate for BERT fine-tuning
WEIGHT_DECAY  = 0.01 # Penalty to prevent overfitting

# Optimizer
optimizer = AdamW(
    model.parameters(),
    lr=LEARNING_RATE,
    weight_decay=WEIGHT_DECAY,
)

# Scheduler
total_steps  = len(train_loader) * NUM_EPOCHS
# Set the first 10% of steps for warmup (helps with training stability at the
start)
warmup_steps = int(0.1 * total_steps)

scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=warmup_steps,
    num_training_steps=total_steps,
)

print(f"Epochs        : {NUM_EPOCHS}")
print(f"Learning rate : {LEARNING_RATE}")
print(f"Total steps   : {total_steps}")
print(f"Warmup steps  : {warmup_steps}")
print("Optimizer and scheduler ready!")

# Step 4 — Training Loop and Validation
# The model passes through the data, calculates its error (Loss), performs
backpropagation, and adjusts its weights. At the end of each epoch, it is
evaluated on the validation set, and the best model is automatically saved to
Drive.

import time

def evaluate(model, loader, device):
    """
    Function to calculate loss and accuracy on a given dataloader.
```

```
    """
    model.eval() # Put the model in evaluation mode (disables Dropout, etc.)
    total_loss, correct, total = 0, 0, 0

    with torch.no_grad(): # Disable gradient calculation to save memory and time
        for batch in loader:
            # Move tensors to the correct device (GPU/CPU)
            input_ids      = batch["input_ids"].to(device)
            attention_mask = batch["attention_mask"].to(device)
            labels         = batch["labels"].to(device)

            # Forward pass: get the raw predictions (logits)
            outputs = model(
                input_ids=input_ids,
                attention_mask=attention_mask,
                labels=labels
            )

            total_loss += outputs.loss.item()
            # Choose the class with the highest probability
            preds       = torch.argmax(outputs.logits, dim=1)
            correct    += (preds == labels).sum().item()
            total      += labels.size(0)

    avg_loss = total_loss / len(loader)
    accuracy = correct / total
    return avg_loss, accuracy

# Dictionary to store the history of metrics (useful for plotting later)
history = {
    "train_loss": [], "train_acc": [],
    "val_loss":   [], "val_acc":  [],
}

best_val_acc  = 0.0
MODEL_SAVE_PATH = "/content/drive/MyDrive/Licenta/best_model"

print("Starting training...\n")

for epoch in range(NUM_EPOCHS):
    model.train() # Activate training mode
    total_loss, correct, total = 0, 0, 0
    start_time = time.time()

    for batch_idx, batch in enumerate(train_loader):
        # 1. Extract data from the batch
        input_ids      = batch["input_ids"].to(DEVICE)
        attention_mask = batch["attention_mask"].to(DEVICE)
        labels         = batch["labels"].to(DEVICE)

        # 2. Clear previously accumulated gradients
```

```
        optimizer.zero_grad()

        # 3. Forward pass (Pass data through the model)
        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )

        loss = outputs.loss

        # 4. Backward pass (Calculate gradients)
        loss.backward()

        # Gradient clipping: Prevents the "exploding gradients" problem
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

        # 5. Update weights and learning rate
        optimizer.step()
        scheduler.step()

        # Calculate live metrics for training
        total_loss += loss.item()
        preds       = torch.argmax(outputs.logits, dim=1)
        correct    += (preds == labels).sum().item()
        total      += labels.size(0)

        # Print progress every 100 batches
        if (batch_idx + 1) % 100 == 0:
            print(f"  Epoch {epoch+1} | Batch {batch_idx+1}/{len(train_loader)} "
                  f"| Loss: {loss.item():.4f}")

    # Calculate final metrics for the epoch
    train_loss = total_loss / len(train_loader)
    train_acc  = correct / total
    val_loss, val_acc = evaluate(model, val_loader, DEVICE)
    elapsed    = time.time() - start_time

    # Save data to history for plotting
    history["train_loss"].append(train_loss)
    history["train_acc"].append(train_acc)
    history["val_loss"].append(val_loss)
    history["val_acc"].append(val_acc)

    print(f"\nResult Epoch {epoch+1}/{NUM_EPOCHS} — {elapsed:.0f}s")
    print(f"  Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f}")
    print(f"  Val Loss:   {val_loss:.4f} | Val Acc:   {val_acc:.4f}")

    # Save the model only if it achieved a better accuracy on the validation set
    if val_acc > best_val_acc:
        best_val_acc = val_acc
```

```
        model.save_pretrained(MODEL_SAVE_PATH)
        print(f"  ☑ Best model saved! Val Acc: {val_acc:.4f}")

    print()

print(f"Training complete! Best Validation Accuracy: {best_val_acc:.4f}")

# Step 5 — Generating Performance Plots
# We save the training history in a plot to observe how the loss function and
accuracy evolved over the epochs.

import matplotlib.pyplot as plt

# Values from our training run
history = {
    "train_loss": [0.2770, 0.1425, 0.0746],
    "val_loss":   [0.1652, 0.2563, 0.2540],
    "train_acc":  [0.8835, 0.9584, 0.9817],
    "val_acc":    [0.9379, 0.9369, 0.9462],
}

NUM_EPOCHS = 3

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

epochs = range(1, NUM_EPOCHS + 1)

# --- Plot 1: Loss ---
axes[0].plot(epochs, history["train_loss"],
             "b-o", label="Train Loss", linewidth=2, markersize=8)
axes[0].plot(epochs, history["val_loss"],
             "r-o", label="Val Loss",   linewidth=2, markersize=8)
axes[0].set_title("Loss per Epoch",  fontsize=14)
axes[0].set_xlabel("Epoch",          fontsize=12)
axes[0].set_ylabel("Loss",           fontsize=12)
axes[0].legend(fontsize=11)
axes[0].set_xticks(epochs)
axes[0].grid(True, alpha=0.3)

# --- Plot 2: Accuracy ---
axes[1].plot(epochs, history["train_acc"],
             "b-o", label="Train Accuracy", linewidth=2, markersize=8)
axes[1].plot(epochs, history["val_acc"],
             "r-o", label="Val Accuracy",   linewidth=2, markersize=8)
axes[1].set_title("Accuracy per Epoch", fontsize=14)
axes[1].set_xlabel("Epoch",             fontsize=12)
axes[1].set_ylabel("Accuracy",          fontsize=12)
axes[1].legend(fontsize=11)
axes[1].set_xticks(epochs)
axes[1].grid(True, alpha=0.3)
```

```
plt.suptitle("BERT Fine-tuning on IMDB — Training History",
             fontsize=15, fontweight="bold", y=1.02)
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Licenta/training_history.png",
            dpi=150, bbox_inches="tight")
plt.show()
print("Plot saved to Drive!")



#
# ANNEX 6: comparison.ipynb — MAX_LENGTH Comparison Experiment
#

# Step 1 — Environment Setup and Global Configurations

# Import operating system and core numerical/tensor libraries
import os, torch, numpy as np
import matplotlib.pyplot as plt
from datasets import load_from_disk
from transformers import (BertTokenizer,
                          BertForSequenceClassification,
                          get_linear_schedule_with_warmup)
from torch.utils.data import DataLoader, TensorDataset
from torch.optim import AdamW
from sklearn.metrics import accuracy_score, f1_score
from google.colab import drive

drive.mount('/content/drive')

DEVICE     = torch.device("cuda" if torch.cuda.is_available() else "cpu")
CSV_PATH   = "/content/drive/MyDrive/Licenta/IMDB Dataset.csv"
MODEL_NAME = "bert-base-uncased"
BATCH_SIZE = 16
NUM_EPOCHS = 3

print(f"Device: {DEVICE}")

# Step 2 — Data Preprocessing and Tokenization Function

# Import pandas for data manipulation and re for regular expressions
import pandas as pd, re
from sklearn.model_selection import train_test_split

def prepare_data(max_length):
    """Tokenize and split dataset for a given max_length."""
    print(f"\nPreparing data with MAX_LENGTH={max_length}...")

    df = pd.read_csv(CSV_PATH) # Load the CSV file into a pandas DataFrame
```

```
    df.columns = ["text", "sentiment"] # Standardize column names
    df["label"] = df["sentiment"].map({"positive": 1, "negative": 0}) # Map
string sentiments to binary integer labels
    df["text"] = df["text"].apply(lambda t: re.sub(r"\s+", " ",  # Clean text:
remove HTML tags, replace multiple spaces with single space, and strip whitespace
                  re.sub(r"<.*?>", " ", t)).strip())
    df = df.drop_duplicates(subset=["text"]).reset_index(drop=True) # Drop
duplicated rows based on text content to prevent data leakage

    # Split the full dataset into train/validation (85%) and test (15%) sets
    train_val, test_df = train_test_split(df, test_size=0.15,
                         random_state=42, stratify=df["label"])

    # Split the train_val set further into train (70% of total) and validation
(15% of total)
    train_df, val_df   = train_test_split(train_val,
                         test_size=0.15/0.85,
                         random_state=42, stratify=train_val["label"])

    tokenizer = BertTokenizer.from_pretrained(MODEL_NAME)

    # Helper function to tokenize text sequences to a specified max_length
    def tokenize(texts):
        return tokenizer(texts, truncation=True,
                         padding="max_length",
                         max_length=max_length,
                         return_tensors="pt")

    # Helper function to create PyTorch DataLoaders from DataFrames
    def make_loader(df, shuffle):
        enc = tokenize(df["text"].tolist()) # Tokenize all text entries in the
dataset split
        labels = torch.tensor(df["label"].tolist(), dtype=torch.long) # Convert
pandas labels to PyTorch tensors
        ds = TensorDataset(enc["input_ids"], # Group input IDs, attention masks,
and labels into a TensorDataset
                           enc["attention_mask"], labels)
        return DataLoader(ds, batch_size=BATCH_SIZE, shuffle=shuffle) # Wrap
dataset in a DataLoader for batching and optional shuffling

    # Return DataLoaders for train (shuffled), validation (unshuffled), and test
(unshuffled) sets
    return (make_loader(train_df, True),
            make_loader(val_df,   False),
            make_loader(test_df,  False))

# Step 3 — Model Training and Evaluation Logic

# Model Training and Validation Logic
# Trains BERT for NUM_EPOCHS and evaluates ONLY on the VALIDATION set.
# The test set is never touched here — it is reserved for evaluate.ipynb.
```

```
# Best model for each MAX_LENGTH is saved to Drive.

import time

def train_and_evaluate(max_length):
    """Fine-tune BERT and evaluate on the validation set only."""
    total_start = time.time()

    # Prepare loaders — test_loader returned but not used here
    train_loader, val_loader, _ = prepare_data(max_length)

    # Load pre-trained BERT for binary classification
    model = BertForSequenceClassification.from_pretrained(
                MODEL_NAME, num_labels=2).to(DEVICE)

    # AdamW optimizer with weight decay
    optimizer   = AdamW(model.parameters(), lr=2e-5, weight_decay=0.01)
    total_steps = len(train_loader) * NUM_EPOCHS
    # Linear warmup over 10% of steps, then linear decay to zero
    scheduler   = get_linear_schedule_with_warmup(
                      optimizer,
                      num_warmup_steps=int(0.1 * total_steps),
                      num_training_steps=total_steps)

    epoch_times  = []
    best_val_acc = 0.0
    best_val_f1  = 0.0

    # Folder where the best model for this MAX_LENGTH will be saved
    save_path = f"/content/drive/MyDrive/Licenta/model_maxlen_{max_length}"
    os.makedirs(save_path, exist_ok=True)

    for epoch in range(NUM_EPOCHS):
        epoch_start = time.time()
        model.train()
        total_loss, correct, total = 0, 0, 0

        # Training loop
        for input_ids, attention_mask, labels in train_loader:
            input_ids      = input_ids.to(DEVICE)
            attention_mask = attention_mask.to(DEVICE)
            labels         = labels.to(DEVICE)

            optimizer.zero_grad()
            outputs = model(input_ids=input_ids,
                            attention_mask=attention_mask,
                            labels=labels)
            outputs.loss.backward()
            # Clip gradients to prevent exploding gradient problem
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            optimizer.step()
```

```
            scheduler.step()

            total_loss += outputs.loss.item()
            preds       = torch.argmax(outputs.logits, dim=1)
            correct    += (preds == labels).sum().item()
            total      += labels.size(0)

        train_acc  = correct / total
        epoch_time = time.time() - epoch_start
        epoch_times.append(round(epoch_time / 60, 1))  # store in minutes

        # Validation loop — no gradient updates
        model.eval()
        val_preds, val_labels_list = [], []
        with torch.no_grad():
            for input_ids, attention_mask, labels in val_loader:
                out   = model(input_ids=input_ids.to(DEVICE),
                              attention_mask=attention_mask.to(DEVICE))
                preds = torch.argmax(out.logits, dim=1)
                val_preds.extend(preds.cpu().numpy())
                val_labels_list.extend(labels.numpy())

        val_acc = accuracy_score(val_labels_list, val_preds)
        val_f1  = f1_score(val_labels_list, val_preds)

        # Save model if it is the best so far for this MAX_LENGTH
        if val_acc > best_val_acc:
            best_val_acc = val_acc
            best_val_f1  = val_f1
            model.save_pretrained(save_path)
            print(f"  ☑ Best model saved at epoch {epoch+1} -> {save_path}")

        print(f"  Epoch {epoch+1}/{NUM_EPOCHS} | "
              f"Train Acc: {train_acc:.4f} | Val Acc: {val_acc:.4f} | "
              f"Val F1: {val_f1:.4f} | Time: {epoch_time/60:.1f} min")

    total_time = round((time.time() - total_start) / 60, 1)
    print(f"\n  MAX_LENGTH={max_length} -> "
          f"Best Val Acc: {best_val_acc:.4f} | Best Val F1: {best_val_f1:.4f} | "
          f"Total time: {total_time:.1f} min")
    print(f"  Model saved to: {save_path}")

    # Return ONLY validation metrics — test set is NEVER touched here
    return best_val_acc, best_val_f1, total_time, epoch_times

# Step 4 — Execute Training Pipeline for Each Sequence Length

# Initialize results dictionary before running individual MAX_LENGTH experiments
results = {}
print("Ready. Run each MAX_LENGTH cell below separately.")
```

```
# 4.1 — MAX_LENGTH = 128

# ── Train and validate with MAX_LENGTH = 128 ──────────────────────────
# Run this cell independently. It trains BERT with MAX_LENGTH=128 and
# evaluates on the validation set only. Does NOT touch the test set.

print("=" * 50)
print("Training with MAX_LENGTH = 128")
print("=" * 50)

val_acc_128, val_f1_128, time_128, epoch_times_128 = train_and_evaluate(128)

results[128] = {
    "val_acc":     val_acc_128,
    "val_f1":      val_f1_128,
    "total_time":  time_128,
    "epoch_times": epoch_times_128
}
print(f"MAX_LENGTH=128 done. Val Acc: {val_acc_128:.4f} | Val F1:
{val_f1_128:.4f}")

# 4.2 — MAX_LENGTH = 256

# ── Train and validate with MAX_LENGTH = 256 ──────────────────────────
# Run this cell independently after MAX_LENGTH=128 finishes.
# Evaluates on the validation set only. Does NOT touch the test set.

print("=" * 50)
print("Training with MAX_LENGTH = 256")
print("=" * 50)

val_acc_256, val_f1_256, time_256, epoch_times_256 = train_and_evaluate(256)

results[256] = {
    "val_acc":     val_acc_256,
    "val_f1":      val_f1_256,
    "total_time":  time_256,
    "epoch_times": epoch_times_256
}
print(f"MAX_LENGTH=256 done. Val Acc: {val_acc_256:.4f} | Val F1:
{val_f1_256:.4f}")

# 4.3 — MAX_LENGTH = 512

# ── Train and validate with MAX_LENGTH = 512 ──────────────────────────
# Run this cell independently after MAX_LENGTH=256 finishes.
# Evaluates on the validation set only. Does NOT touch the test set.

print("=" * 50)
print("Training with MAX_LENGTH = 512")
print("=" * 50)
```

```
val_acc_512, val_f1_512, time_512, epoch_times_512 = train_and_evaluate(512)

results[512] = {
    "val_acc":     val_acc_512,
    "val_f1":      val_f1_512,
    "total_time":  time_512,
    "epoch_times": epoch_times_512
}
print(f"MAX_LENGTH=512 done. Val Acc: {val_acc_512:.4f} | Val F1:
{val_f1_512:.4f}")

# Step 5 — Select Best Configuration Based on Validation Performance

results = {
    128: {"val_acc": 0.8990, "val_f1": 0.9006, "total_time":
50.9,  "epoch_times": [15.2, 15.4, 15.4]},
    256: {"val_acc": 0.9277, "val_f1": 0.9280, "total_time":
97.4,  "epoch_times": [30.0, 29.9, 30.0]},
    512: {"val_acc": 0.9426, "val_f1": 0.9433, "total_time": 174.1,
"epoch_times": [53.8, 53.8, 53.8]},
}

best_max_length = max(results, key=lambda k: results[k]["val_acc"])
print(f"Best MAX_LENGTH based on validation : {best_max_length}")
print(f"Val Accuracy                        :
{results[best_max_length]['val_acc']*100:.2f}%")
print(f"Val F1 Score                        :
{results[best_max_length]['val_f1']*100:.2f}%")
print(f"\n-> MAX_LENGTH={best_max_length} selected for final test evaluation in
evaluate.ipynb")

# Select the best MAX_LENGTH configuration based on VALIDATION accuracy only.
# This choice determines which configuration is used for final test evaluation
# in evaluate.ipynb. The test set is used exactly once, there.

best_max_length = max(results, key=lambda k: results[k]["val_acc"])
print(f"Best MAX_LENGTH based on validation accuracy : {best_max_length}")
print(f"Val Accuracy                                 :
{results[best_max_length]['val_acc']*100:.2f}%")
print(f"Val F1 Score                                 :
{results[best_max_length]['val_f1']*100:.2f}%")
print(f"\n-> MAX_LENGTH={best_max_length} selected for final test evaluation in
evaluate.ipynb")

# Step 6 - Generate Comparison Tables and Performance Plots

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
```

```
from google.colab import drive
drive.mount('/content/drive')

# Build comparison table
rows = []
for ml, r in results.items():
    rows.append({
        "MAX_LENGTH":       ml,
        "Val Accuracy (%)": f"{r['val_acc']*100:.2f}%",
        "Val F1 Score (%)": f"{r['val_f1']*100:.2f}%",
        "Epoch 1 Time":     f"{r['epoch_times'][0]:.1f} min",
        "Epoch 2 Time":     f"{r['epoch_times'][1]:.1f} min",
        "Epoch 3 Time":     f"{r['epoch_times'][2]:.1f} min",
        "Total Train Time": f"{r['total_time']:.1f} min",
        "Selected":         "☑ BEST" if ml == best_max_length else ""
    })

df_results = pd.DataFrame(rows)
print("\n" + "="*75)
print("   COMPARISON TABLE — MAX_LENGTH IMPACT (evaluated on Validation Set)")
print("="*75)
print(df_results.to_string(index=False))
print("="*75)

# Save table to Drive
os.makedirs("/content/drive/MyDrive/Licenta", exist_ok=True)
df_results.to_csv("/content/drive/MyDrive/Licenta/comparison_table.csv",
index=False)

# Plot
labels = [str(k) for k in results.keys()]
accs   = [v["val_acc"]   for v in results.values()]
f1s    = [v["val_f1"]    for v in results.values()]
times  = [v["total_time"] for v in results.values()]

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

x     = np.arange(len(labels))
width = 0.35
bars1 = axes[0].bar(x - width/2, accs, width, label="Val Accuracy",
color="steelblue")
bars2 = axes[0].bar(x + width/2, f1s,  width, label="Val F1 Score",
color="coral")
axes[0].set_title("Validation Accuracy & F1 vs MAX_LENGTH", fontsize=13)
axes[0].set_xlabel("MAX_LENGTH (tokens)")
axes[0].set_ylabel("Score")
axes[0].set_xticks(x)
axes[0].set_xticklabels(labels)
axes[0].set_ylim(0.87, 0.97)
axes[0].legend()
axes[0].grid(axis="y", alpha=0.3)
```

```
for bar in list(bars1) + list(bars2):
    axes[0].text(bar.get_x() + bar.get_width()/2,
                 bar.get_height() + 0.001,
                 f"{bar.get_height():.4f}",
                 ha="center", va="bottom", fontsize=9)

axes[1].bar(labels, times, color="mediumseagreen")
axes[1].set_title("Total Training Time vs MAX_LENGTH", fontsize=13)
axes[1].set_xlabel("MAX_LENGTH (tokens)")
axes[1].set_ylabel("Time (minutes)")
axes[1].grid(axis="y", alpha=0.3)
for i, t in enumerate(times):
    axes[1].text(i, t + 1, f"{t:.1f} min", ha="center", fontsize=10)

plt.suptitle("BERT Fine-tuning — MAX_LENGTH Comparison (Validation Set)",
             fontsize=15, fontweight="bold")
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Licenta/comparison_maxlength.png", dpi=150)
plt.show()
print("Saved to Drive!")



#
================================================================================
# ANNEX 7: evaluate.ipynb — Model Evaluation and Error Analysis
#
================================================================================

# BERT Model Evaluation & Inference
# Testing the fine-tuned `bert-base-uncased` model on unseen IMDB reviews

# Step 1 — Imports and Global Variables
# Importing the necessary libraries for data manipulation, model loading, and
defining the paths for our saved datasets and fine-tuned model.

!pip install -q --upgrade datasets # Upgrade the 'datasets' library quietly (-q)
to ensure compatibility

# Import standard libraries for system operations and array manipulation
import matplotlib.pyplot as plt
import seaborn as sns
import torch
import numpy as np
import pandas as pd
import re
from transformers import BertForSequenceClassification, BertTokenizer
from torch.utils.data import DataLoader, TensorDataset
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score,
```

```
    recall_score, f1_score, confusion_matrix,
    classification_report
)

from google.colab import drive
drive.mount('/content/drive')

DEVICE      = torch.device("cuda" if torch.cuda.is_available() else "cpu")
CSV_PATH    = "/content/drive/MyDrive/Licenta/IMDB Dataset.csv"
MAX_LENGTH  = 512
BATCH_SIZE  = 16

# ============================================================================
# LOAD MODEL — best model from comparison experiment
# ============================================================================
MODEL_SAVE_PATH = "/content/drive/MyDrive/Licenta/model_maxlen_512"
print(f"Loading model from: {MODEL_SAVE_PATH}")
model = BertForSequenceClassification.from_pretrained(MODEL_SAVE_PATH)
model = model.to(DEVICE)
model.eval()

# ============================================================================
# LOAD DATA — same pipeline as comparison.ipynb to ensure identical test split
# ============================================================================
print("Loading and preparing data from CSV...")
df = pd.read_csv(CSV_PATH)
df.columns = ["text", "sentiment"]
df["label"] = df["sentiment"].map({"positive": 1, "negative": 0})
df["text"]  = df["text"].apply(lambda t: re.sub(r"\s+", " ",
              re.sub(r"<.*?>", " ", t)).strip())
df = df.drop_duplicates(subset=["text"]).reset_index(drop=True)

# Same split as comparison.ipynb — random_state=42 ensures identical test set
train_val, test_df = train_test_split(df, test_size=0.15,
                     random_state=42, stratify=df["label"])
train_df, val_df   = train_test_split(train_val, test_size=0.15/0.85,
                     random_state=42, stratify=train_val["label"])

print(f"Test samples: {len(test_df)}")

# Tokenize test set
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
enc = tokenizer(
    test_df["text"].tolist(),
    truncation=True,
    padding="max_length",
    max_length=MAX_LENGTH,
    return_tensors="pt"
)

input_ids      = enc["input_ids"]
```

```
attention_mask = enc["attention_mask"]
labels         = torch.tensor(test_df["label"].tolist(), dtype=torch.long)

test_dataset = TensorDataset(input_ids, attention_mask, labels)
test_loader  = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

# Save test texts for error analysis
test_texts = test_df["text"].tolist()

print(f"Test batches : {len(test_loader)}")
print("Ready!")

# Step 2 — Batch Evaluation on Test Data
# Iterating through the `test_loader` to gather model predictions for the entire
test dataset without updating the model weights.

# Initialize empty lists to store all model predictions and true labels
all_preds  = []
all_labels = []

# Disable gradient calculation to save memory and speed up evaluation
with torch.no_grad():
    for input_ids, attention_mask, labels in test_loader:
        input_ids      = input_ids.to(DEVICE)
        attention_mask = attention_mask.to(DEVICE)

        outputs = model(input_ids=input_ids, attention_mask=attention_mask) #
Forward pass: get the model's output (logits) for the current batch
        preds   = torch.argmax(outputs.logits, dim=1) # Get the predicted class
by finding the index with the highest logit value

# Convert predictions and labels to NumPy arrays and append to our master lists
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.numpy())

print(f"Done! {len(all_preds)} predictions")

# Step 3 — Classification Report and Confusion Matrix Visualization

accuracy  = accuracy_score(all_labels, all_preds)
precision = precision_score(all_labels, all_preds)
recall    = recall_score(all_labels, all_preds)
f1        = f1_score(all_labels, all_preds)

print("=" * 45)
print("         FINAL TEST SET RESULTS")
print("=" * 45)
print(f"  Accuracy  : {accuracy:.4f}  ({accuracy*100:.2f}%)")
print(f"  Precision : {precision:.4f}  ({precision*100:.2f}%)")
print(f"  Recall    : {recall:.4f}  ({recall*100:.2f}%)")
print(f"  F1 Score  : {f1:.4f}  ({f1*100:.2f}%)")
```

```
print("=" * 45)

print("\nDetailed Classification Report:")
print(classification_report(
    all_labels, all_preds,
    target_names=["Negative", "Positive"]
))

cm = confusion_matrix(all_labels, all_preds) # Calculate the confusion matrix
based on the true labels and the model's predictions

plt.figure(figsize=(8, 6))
# Create a heatmap using seaborn to visualize the confusion matrix
sns.heatmap(
    cm,
    annot=True, # displays the numbers in the cells
    fmt="d", # formats them as integers
    cmap="Blues",
    xticklabels=["Negative", "Positive"],
    yticklabels=["Negative", "Positive"],
)
plt.title("Confusion Matrix — BERT on IMDB Test Set", fontsize=14, pad=15)
plt.ylabel("True Label", fontsize=12)
plt.xlabel("Predicted Label", fontsize=12)
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Licenta/confusion_matrix.png", dpi=150)
plt.show()
print("Confusion matrix saved to Drive!")

# Step 4 — Data Preparation and Dataset Alignment

# Load test texts from the pre-saved CSV file.
# This file was created during preprocessing and contains the EXACT texts
# from the same test split used during training — ensuring correct alignment.
# Using test_texts.csv avoids re-running the split and guarantees consistency.

import pandas as pd
import os

TEST_TEXTS_PATH = "/content/drive/MyDrive/Licenta/test_texts.csv"

# Load the test texts CSV saved during preprocessing
test_df    = pd.read_csv(TEST_TEXTS_PATH)
test_texts = test_df["text"].tolist()

print(f"Test texts loaded from CSV : {len(test_texts)}")
print(f"First review preview       : {test_texts[0][:150]}...")

# Step 5 - Calculate Evaluation Metrics and Error Rates
```

```
wrong_indices  = [i for i, (p, l) in enumerate(zip(all_preds, all_labels)) if p
!= l] # Find the indices where the prediction (p) does not match the actual label
(l)
correct_indices = [i for i, (p, l) in enumerate(zip(all_preds, all_labels)) if p
== l] # Find the indices where the prediction (p) correctly matches the actual
label (l)

# Map numerical labels to their human-readable string representations
label_map = {0: "Negative", 1: "Positive"}

print(f"Total errors   : {len(wrong_indices)} / {len(all_labels)}")
print(f"Error rate     : {len(wrong_indices)/len(all_labels)*100:.2f}%")
print(f"Correct        : {len(correct_indices)}")

# Step 6 - Display Misclassified Examples

print("=" * 70)
print("MISCLASSIFIED EXAMPLES")
print("=" * 70)

# Iterate through the first 5 indices of incorrectly predicted examples
for idx in wrong_indices[:5]:
    text       = test_texts[idx]
    true_label = label_map[all_labels[idx]]
    pred_label = label_map[all_preds[idx]]

    print(f"\nExample #{idx}")
    print(f"  True Label : {true_label}")
    print(f"  Predicted  : {pred_label}")
    print(f"  Text       : {text[:300]}...")
    print("-" * 70)

for idx in wrong_indices[:10]:
    print(f"\nExample #{idx}")
    print(f"True: {label_map[all_labels[idx]]}")
    print(f"Predicted: {label_map[all_preds[idx]]}")
    print(f"Text: {test_texts[idx][:400]}")
    print("-" * 70)

# Step 7 - Analyze and Plot Error Distribution by Review Length

# Check if longer reviews have more errors
wrong_lengths   = [len(test_texts[i].split()) for i in wrong_indices]
correct_lengths = [len(test_texts[i].split()) for i in correct_indices]

print(f"Average length of WRONG predictions   :
{sum(wrong_lengths)/len(wrong_lengths):.0f} words")
print(f"Average length of CORRECT predictions :
{sum(correct_lengths)/len(correct_lengths):.0f} words")

# Plot 1 — Misclassified reviews only
```

```
plt.figure(figsize=(10, 4))
plt.hist(wrong_lengths, bins=50, color="red", alpha=0.8)
plt.xlabel("Review length (words)", fontsize=12)
plt.ylabel("Count", fontsize=12)
plt.title("Review Length Distribution — Misclassified Reviews", fontsize=14)
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Licenta/error_length_misclassified.png",
dpi=150)
plt.show()
print("Misclassified plot saved!")

# Plot 2 — Correct reviews only
plt.figure(figsize=(10, 4))
plt.hist(correct_lengths, bins=50, color="blue", alpha=0.8)
plt.xlabel("Review length (words)", fontsize=12)
plt.ylabel("Count", fontsize=12)
plt.title("Review Length Distribution — Correctly Classified Reviews",
fontsize=14)
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Licenta/error_length_correct.png", dpi=150)
plt.show()
print("Correct plot saved!")

# Step 8 - Analyze Impact of Irony and Sarcasm on Model Performance

# Define a list of keywords commonly associated with irony or sarcasm
irony_keywords = [
    "supposedly", "apparently", "brilliant", "genius",
    "masterpiece", "wow", "amazing", "sure", "right",
    "of course", "obviously", "great job", "well done"
]

# Initialize empty lists to store indices of reviews with potential irony
irony_errors   = []
irony_correct  = []

# Iterate through indices of incorrectly classified reviews (errors)
for idx in wrong_indices:
    text_lower = test_texts[idx].lower() # Convert text to lowercase for case-
insensitive keyword matching
    if any(kw in text_lower for kw in irony_keywords): # Check if any of the
predefined irony keywords exist in the text
        irony_errors.append(idx)

# Iterate through indices of correctly classified reviews
for idx in correct_indices:
    text_lower = test_texts[idx].lower() # Convert text to lowercase for case-
insensitive keyword matching
    if any(kw in text_lower for kw in irony_keywords): # Check if any of the
predefined irony keywords exist in the text
        irony_correct.append(idx)
```

```
# Calculate the total number of reviews containing potential irony keywords
total_irony = len(irony_errors) + len(irony_correct)
print(f"Reviews with potential irony/sarcasm keywords : {total_irony}")
print(f"  Misclassified : {len(irony_errors)} "
      f"({len(irony_errors)/total_irony*100:.1f}%)")
print(f"  Correct       : {len(irony_correct)} "
      f"({len(irony_correct)/total_irony*100:.1f}%)")

print("\nExample of potentially ironic misclassified review:")
if irony_errors:
    idx  = irony_errors[0]
    print(f"  True: {label_map[all_labels[idx]]} | "
          f"Predicted: {label_map[all_preds[idx]]}")
    print(f"  Text: {test_texts[idx][:300]}...")

# Step 9 — Save Final Metrics to CSV
# Saving all evaluation metrics and hyperparameters to a CSV file on Google Drive
for future reference.

# Save final test set metrics to Google Drive.
# Drive is already mounted in Step 1 — os.makedirs ensures folder exists.

import os
import pandas as pd

os.makedirs("/content/drive/MyDrive/Licenta", exist_ok=True)

metrics = {
    "Accuracy":      [round(accuracy, 4)],
    "Precision":     [round(precision, 4)],
    "Recall":        [round(recall, 4)],
    "F1 Score":      [round(f1, 4)],
    "MAX_LENGTH":    [512],
    "Epochs":        [3],
    "Batch Size":    [16],
    "Learning Rate": [2e-5],
}

pd.DataFrame(metrics).to_csv(
    "/content/drive/MyDrive/Licenta/final_metrics.csv", index=False
)
print("Metrics saved to Drive!")



#
================================================================================
# ANNEX 8: app.py — Gradio Web Interface
#
================================================================================
```

```python
# ============================================================================
# app.py
# Gradio web interface for BERT-based sentiment analysis
# Run: python app.py
# ============================================================================

import os

# Fix for Windows OpenMP conflict
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
os.environ["OMP_NUM_THREADS"]      = "1"

import torch
import gradio as gr
from transformers import BertTokenizer, BertForSequenceClassification

# ============================================================================
# CONFIGURATION
# ============================================================================

MODEL_PATH = "best_model"       # path to saved fine-tuned BERT model
MAX_LENGTH = 512                # must match the value used during training
DEVICE     = torch.device("cpu") # CPU is sufficient for inference

# ============================================================================
# LOAD MODEL AND TOKENIZER
# ============================================================================

print("Loading tokenizer...")
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

print("Loading model...")
model = BertForSequenceClassification.from_pretrained(MODEL_PATH)
model.to(DEVICE)
model.eval()   # set to evaluation mode — disables dropout
print("Model ready!\n")

# ============================================================================
# PREDICTION FUNCTION
# ============================================================================

def predict(text):
    """
    Takes any input text and returns:
      - sentiment label: POSITIVE or NEGATIVE
      - confidence score as a percentage
    """
    if not text.strip():
        return " Please enter some text.", "N/A"
```

```
    # Tokenize input text
    inputs = tokenizer(
        text,
        truncation=True,        # cut sequences longer than MAX_LENGTH
        padding="max_length",   # pad shorter sequences to MAX_LENGTH
        max_length=MAX_LENGTH,
        return_tensors="pt"
    )

    # Run inference — no gradient needed for prediction
    with torch.no_grad():
        outputs = model(
            input_ids=inputs["input_ids"].to(DEVICE),
            attention_mask=inputs["attention_mask"].to(DEVICE)
        )

    # Convert logits to probabilities
    probs      = torch.softmax(outputs.logits, dim=1)[0]
    pred_label = torch.argmax(probs).item()
    confidence = probs[pred_label].item()

    # Map label index to human-readable string
    # 0 = Negative, 1 = Positive (IMDB dataset convention)
    label = "POSITIVE " if pred_label == 1 else "NEGATIVE "

    return label, f"{confidence * 100:.2f}%"

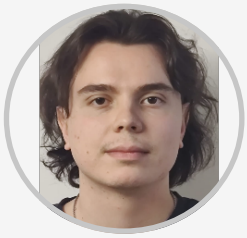
# ================================================================================
# GRADIO INTERFACE
# ================================================================================

demo = gr.Interface(
    fn=predict,
    inputs=gr.Textbox(
        lines=6,
        placeholder="Enter any text here and click Submit...",
        label="Input Text"
    ),
    outputs=[
        gr.Textbox(label="Sentiment"),
        gr.Textbox(label="Confidence Score")
    ],
    title=" BERT Sentiment Analyzer",
    description=(
        "This application uses a fine-tuned **BERT** (bert-base-uncased) model "
        "trained on the **IMDB movie reviews dataset** (50,000 reviews) to
predict "
        "whether a given text expresses a **Positive** or **Negative**
sentiment.\n\n"
        "**Model accuracy: 94.14%** on the test set."
    ),
```

```
    examples=[
        ["This movie was absolutely amazing! The acting was superb and the plot
kept me on the edge of my seat."],
        ["Terrible film. Complete waste of time. Boring, predictable and poorly
written."],
        ["It was an average movie, nothing special but not terrible either."],
        ["One of the best performances I have ever seen. A true masterpiece of
cinema."],
        ["I fell asleep halfway through. Very disappointing given the great
reviews."]
    ],
    theme=gr.themes.Soft(),
    allow_flagging="never"
)

# Launch the app
# share=True generates a public link valid for 72 hours
demo.launch(share=True)
```

europass



Morar Horea-Răzvan

Work permit: Romanian | **Date of birth:** 08/09/2003 | **Place of birth:** Zalau, Romania | **Nationality:** Romanian | **Sex:** Male | **Phone:** (+40) 757217620 (Mobile) | **Email:** morarhorea@yahoo.com | **Address:** str Arinei,15, 450077, Zalau, Romania (Home) | **Address:** Observatorului, 34, 400489, Cluj-Napoca, Romania (Work)

WORK EXPERIENCE

S.C.ELECTRO GROUP S.R.L – ZALAU, ROMANIA

UNSKILLED ASSEMBLY AND INSTALLATION WORKER – 01/08/2024 – 31/08/2024

- Assisted in the maintenance and replacement of outdoor lighting systems,under supervision.
-Supported electrical technicians in disassembling, cleaning, and reassembling industrial equipment in factory settings.
-Participated in basic cable management tasks during equipment interventions and system maintenance.
-Contributed to the disassembly and reassembly of electrical panels, ensuring proper alignment and cleanliness of components.

EDUCATION AND TRAINING

01/10/2022 – CURRENT Cluj-Napoca, Romania

BACHELORS IN ENGINEERING Universitatea Tehnica Cluj-Napoca

Address Strada Memorandumului 28, 400114, Cluj-Napoca, Romania | **Website** https://www.utcluj.ro/

01/09/2018 – 01/06/2022 Zalau, Romania

HIGH SCHOOL DIPLOMA Colegiul National "Silvania"

Website https://cnszalau.ro/

LANGUAGE SKILLS

Mother tongue(s): **ROMANIAN**

Other language(s):

	UNDERSTANDING		SPEAKING		WRITING
	Listening	Reading	Spoken production	Spoken interaction	
ENGLISH	B2	B2	C1	C2	B2

Levels: A1 and A2: Basic user - B1 and B2: Independent user - C1 and C2: Proficient user

SKILLS

Circuit Design: LTspice | Microsoft Excel | Microsoft Word | Microsoft Powerpoint

Programming Languages

SQL | Java | JavaScript | Matlab | HTML | C++ | VHDL language(basic)

ORGANISATIONAL SKILLS

Organisational Abilities

Ability to work under pressure
Time Management
Adaptable

Teamwork and Collaboration Skills
Quick Learner

HOBBIES AND INTERESTS

Creative and Outdoor Activities

I enjoy playing basketball and participating in team sports to develop coordination and teamwork, listening to music for relaxation and inspiration, and reading to expand my knowledge, especially in technical and practical fields

PROJECTS

Four Stage Analog Signal Processor

Designed in CAD software a circuit that processes an analog signal in 4 steps:
1. A passive current feedback instrumentation amplifier
2. Analog Low-Pass Filter (KHN)
3.A programmable gain amplifier with RG Parallel
4. Precision full-wave rectifier

Link : https://github.com/MorarHorea/Four-Stage-Analog-Signal-Processor

Voltage Stabilizer with Zener Diode

In this project we have a Parametric Voltage Stabilizer which maintains constant output voltage within specified limits despite variations in input voltage, load current, and temperature,that are represented in graphics.

Link : https://github.com/MorarHorea/Voltage-Stabilizer-with-Zener-Diode

Security access reader for entering building using microcontrollers

In this project we have a security access reader for entering buildings using microcontrollers constructed with a single-board Arduino Uno and a RFID technology for user identification.In addition a code for the Arduino is used for user registration,access restriction and access monitoring by recording data logs into database.

Link : https://github.com/MorarHorea/Security-access-reader-for-entering-building-using-microcontrollers

Circuit-for-controlling-soil-moisture-level-for-plants

Designed in CAD software a circuit-for-controlling-soil-moisture-level-for-plants.This system is an automated irrigation controller that uses resistive soil moisture sensors and a hysteresis comparator to maintain soil humidity within a specified range.This setup ensures automatic and stable watering, avoiding rapid switching (thanks to hysteresis), and gives a visual status indication via the LED.

Link : https://github.com/MorarHorea/Circuit-for-controlling-soil-moisture-level-for-plants

VOLUNTEERING

01/11/2024 – CURRENT
Volunteer – OSSUB (Student Organization)

-Coordinated the organization of a street basketball competition, handling logistics, scheduling, and team coordination
-Participated in a debate activity, representing the organization and practicing public speaking and argumentation skills

EuroJunior competition

Won 1st place nationally at the EuroJunior competition in first grade, reflecting early initiative and excellence

High School Athletic Achievement

Represented my high school in the basketball team, achieving 3rd place at the county-level (județene) championship



Conform cu originalul